TECHNISCHE HOCHSCHULE DEGGENDORF
Campus Cham

Master's Thesis

Deggendorf Institute of Technology, Campus Cham
Faculty of Applied Natural Sciences and Industrial Engineering
Master Mechatronics & Cyber-Physical Systems

Automatische Programmerzeugung für einen Blechhandhabungsroboter vor einer Abkantpresse

Automatic Program Generation for a Sheet-Metal Handling Robot in Front of a Press Brake

Master's thesis to obtain the academic degree:
Master of Engineering (M.Eng.)

Submitted by: Sooraj Sudhakaran.
Matriculation Number: XXXXXXXX
First Examiner: Prof. Dr. Matthias Górka

Cham, April 2, 2026

Abstract

Acknowledgements

List of Tables

3.1 Functional Requirements for Automatic Robot Program Generation 23
3.2 Non-Functional Requirements — NFR-01 to NFR-04 with acceptance criteria and verification methods 25
3.3 Requirement Traceability Matrix — all requirements (FR-00 to FR-07, NFR-01 to NFR-05, SR-01 to SR-03) mapped to implementation modules and verification scenarios 31

4.1 Structural comparison of Concept A, Concept B, and Concept C across key design features 41
4.2 Weighted Sum Model (WSM) evaluation of Concept A, Concept B, and Concept C 45

5.1 Software modules across system layers and their primary responsibilities 51
5.2 Failure Mode and Effects Analysis (FMEA) of the robotic bending system 83

List of Figures

2.1 Robotic press brake bending system with integrated robot handling source -[7] 3
2.2 Teach pendant interface for manual robot programming (Source: Own work9) 6
2.3 Offline robot programming environment with CAD-based simulation (Source: [14]) 8
2.4 Robotic sheet metal bending system with integrated robot and press brake ([19]) 12

3.1 Tool Center Point parameters in Drag&Bot. 29

4.1 Manual robot program construction in the drag&bot environment using function blocks for motion and gripper control. . . . 33
4.2 Manual robot program construction in the drag&bot environment using function blocks for motion and gripper control. . . . 34
4.3 Virtual robotic environment used to evaluate the proposed concept (Drag & Bot platform). 39

5.1 System architecture 48
5.2 Five layer architecture 50
5.3 Communication architecture of the implemented system, illustrating the interaction between the planning software, the drag&bot runtime environment, and the robot controller (adapted from [20]). 54
5.4 Robot control panel interface used for system operation and monitoring. 55
5.5 Operator workflow illustrating the sequential execution stages. Each stage must be completed before the next begins. All operator interactions occur within the drag&bot environment. *Source: own work.* 58
5.6 Sheet metal workpiece model used for geometry analysis. Internal cut-outs are detected and excluded from the grip candidate search region. Source: own work. 59

5.7 Operator Workflow .Every gate must be passed before the next begins.All operator interactions occu within the drag and bot enviornment. *Source: own work.* 60
5.8 Tool Centre Point (TCP) configuration of the UR5e robot used for motion planning. *Source: own work.* 62
5.9 Spatial zone classification of the robot workspace based on the sheet centroid position. *Source: own work.* 63
5.10 Fixture pose deviation validation workflow. Each monitored fixture is evaluated by comparing positional and angular deviations against defined thresholds. Out-of-tolerance or missing fixtures trigger a blocking warning dialog requiring operator confirmation. Validation is applied uniformly across all spatial zones. Source: own work. 65
5.11 Three-level orientation selection strategy for determining a valid pickup pose under kinematic constraints. *Source: own work.* 69
5.12 Grip position planning and feasibility validation workflow. The process integrates geometric filtering and vacuum force validation to ensure physically stable and executable grasp configurations prior to trajectory generation. Source: own work. 73
5.13 Opposite-face flip workflow enabling access to previously unreachable bending surfaces. Source: own work. 77

List of Abbreviations

OLP	:	Offline Programming
ROS	:	Robot Operating System
TCP	:	Tool Center Point
UI	:	User Interface
CAD	:	Computer-Aided Design
CAM	:	Computer-Aided Manufacturing
API	:	Application Programming Interface
DOF	:	Degree of Freedom
FR	:	Functional Requirements
NFR	:	Non-Functional Requirements
WSM	:	Weighted Sum Model
URDF	:	Unified Robot Description Format
YAML	:	YAML Ain’t Markup Language
Digital IO	:	Digital Input/Output
JSON	:	JavaScript Object Notation
SHA-512	:	Secure Hash Algorithm 512-bit
X	:	Translation along the X-axis (metres)
Y	:	Translation along the Y-axis (metres)
Z	:	Translation along the Z-axis (metres)
RX	:	Rotation about the X-axis (degrees)
RY	:	Rotation about the Y-axis (degrees)
RZ	:	Rotation about the Z-axis (degrees)
WebSocket	:	Web-based communication protocol for real-time data exchange defined in RFC 6455
LIN	:	Linear motion command
PTP	:	Point-to-point motion command
HI Dialog	:	Human Interaction (Drag&Bot-specific dialog service)
HTTP	:	Hypertext Transfer Protocol
HTTPS	:	Hypertext Transfer Protocol Secure
IP	:	Internet Protocol
DNB	:	Drag&Bot
UR5e	:	Universal Robots 5e (robot model)
STL	:	Standard Tessellation Language.

Contents

Abstract **I**

Acknowledgements **III**

List of Tables **V**

List of Figures **VIII**

List of Abbreviations **IX**

1 Introduction **1**
1.1 head of robot 1

2 State of the Art and Literature Review **2**
2.1 Fundamental of Industrial Robotics 2
2.2 Sheet Metal Bending and Press Brake systems 3
2.2.1 Process Characteristics and Challenges 3
2.2.2 Grasping and Vacuum-Based Handling 4
2.2.3 Trajectory Planning in Bending Environments 5
2.2.4 Limitations of Existing Industrial Systems 5
2.2.5 Relevance to This Thesis 5
2.3 Manual and Teach Pendant Programming 6
2.3.1 Limitations of Manual Programming 6
2.3.2 Challenges in Complex Manipulation Tasks 7
2.3.3 Transition Towards Automated Programming 7
2.4 Offline Programming Approaches for Industrial Robots 7
2.4.1 Digital Twin and Simulation-Based Programming 8
2.4.2 Limitations of Offline Programming 9
2.5 Automatic Robot Program Generation and Rule-Based Approaches 9
2.5.1 Task-Level Programming and Abstraction 10
2.5.2 Rule-Based and Constraint-Based Planning 10
2.6 Robotic Sheet Metal Bending System 11
2.6.1 Regrip Planning in Sheet Metal Bending 12
2.6.2 Challenges of Regrip Integration in Industrial Systems . . 13
2.6.3 Grasp Planning for Sheet Metal Handling 14
2.6.4 Trajectory Planning in Robotic Bending Systems 14

2.6.5 Limitations of Existing Robotic Bending Systems 15
2.7 Industrial Software Platforms and System Implementations . . 16
2.7.1 Simulation and Offline Programming Environments 16
2.7.2 Graphical Programming and Integrated Execution Systems 17
2.7.3 Relevance to This Thesis 17
2.8 Research Gap and Motivation 18

3 System Concept and Methodology 20
3.1 Problem Definition and Objectives 20
3.2 Requirement Definition and Management 21
3.2.1 Functional Requirements 22
3.2.2 Non-functional Requirements 23
3.2.3 System Constraints and Assumptions 24
3.2.4 Software Platform and Tool requirement 26
3.2.4.1 SR-01 Virtual Execution Environment 27
3.2.4.2 SR-02 Communication Interface Requirement . . 28
3.2.4.3 Tool Centre Point (TCP) Definition Requirement - SR-03 28
3.2.5 System Traceability 29

4 Development of Concepts 32
4.1 Purpose and scope of concept development 32
4.2 Concept A – Structured operator Workflow with Guided Teach in programming 32
4.2.1 Methodological Framework 32
4.2.2 System Structure 34
4.2.3 Critical Evaluation 34
4.3 Concept B – Automatic Program Generation based on rules with Integrated Constraint Validation. 35
4.3.1 Methodological Framework 35
4.3.2 system structure 37
4.3.3 Critical evaluation 38
4.4 Concept C – CAD Integrated Offline Programming with Digital Twin Simulation 38
4.4.1 Methodological Framework 38
4.4.2 System Structure 40
4.4.3 Critical Evaluation 40
4.5 Structural Comparison of Concepts 41
4.6 Concept Decision-Weighed Sum Model (WSM) 42
4.6.1 Methodology 42

4.6.2 Evaluation Criteria and Weights 43
4.7 Result of the weighted Sum Model 44
4.8 Analysis of results and Concept Decision 44

5 Implementation 46
5.1 Introduction and scope 46
5.2 System architecture 48
5.2.1 Layered Architecture Design and data flow 48
5.2.2 Module Overview and dependency Discipline 51
5.2.3 Communication Stack, Authentication interface Specification 53
5.2.4 Configuration Architecture and YAML File Structure 56
5.2.5 Operator interface – interactive 3d sheet bending viewer 57
5.3 Development and Simulation Environment 60
5.3.1 Simulation Platform, Robot Model, and TCP Configuration 60
5.3.2 Virtual Cell Layout and Spatial Zone Classification 61
5.4 Pre-Execution Safety Validation 63
5.4.1 Fixture Pose Deviation Validation 64
5.4.2 Operator Authorization and Runtime Error Monitoring . . 65
5.5 Workpiece Orientation Strategy 66
5.5.1 Zone-Based RZ Admissibility 66
5.5.2 Pickup Orientation Validation 66
5.5.3 Three-Level Orientation Selection Strategy 67
5.6 Workpiece Handling and Gripper Control 68
5.6.1 Vacuum Gripper Digital IO Sequence 69
5.6.2 Kinematic Object Attachment and Detachment 70
5.7 Grip Position Planning and Feasibility Validation 70
5.7.1 Sheet Geometry and Hole Detection 71
5.7.2 Vacuum Holding Force Validation 71
5.7.3 Grip Candidate Search and Selection 72
5.8 Regrip Planning and Execution 73
5.8.1 Regrip Decision Strategy 74
5.8.2 Regrip Execution Sequence 75
5.8.3 System Integration and Configuration 75
5.9 Opposite-face Bending Workflow 76
5.9.1 Pre-Flip Verification and Transition Correction 76
5.9.2 Sheet Flip Sequence and Opposite-face Bending Execution 76
5.10Trajectory Generation 77
5.10.1Three-Phase LIN Trajectory Structure 78

5.10.2 Waypoint Generation and Interpolation 78
5.10.3 Kinematic Mapping and Joint Execution 79
5.10.4 Workspace Constraints and Singularity Handling 79
5.10.5 Motion Execution and Path Reduction 79
5.11 System Integration, Execution Pipeline, and Conclusion 80
5.11.1 Conclusion 81
5.12 System Reliability and Failure Mode Analysis 81
5.12.1 Identified Failure Modes 82
5.12.2 Mitigation Strategies 84
5.12.3 Discussion 84

References **85**

1 Introduction

YSBASHFKJSDHFKfdghghjg

1.1 head of robot

XGFGF sooraj sudhakaran

2 State of the Art and Literature Review

2.1 Fundamental of Industrial Robotics

Industrial robots play a central role in modern manufacturing systems by enabling high precision, repeatability, and efficiency in automated processes. According to the international standard ISO 8373, an industrial robot is defined as a reprogrammable, multipurpose manipulator programmable in three or more axes [1]. These systems are widely used across applications such as material handling, assembly, welding, and forming processes, including sheet metal bending. A typical industrial robot consists of multiple joints that provide degrees of freedom (DOF), allowing the manipulator to position its end-effector in three-dimensional space. The position and orientation of the end-effector are defined using the Tool Center Point (TCP), which represents the reference point for task execution. The relationship between joint variables and the TCP pose is described through forward and inverse kinematics, which form the mathematical foundation for robot motion planning and control [2]. From a programming perspective, industrial robots can be controlled at different levels of abstraction. At the lowest level, motion is defined explicitly in joint space or Cartesian space by specifying waypoints and trajectories. This approach requires detailed knowledge of robot kinematics and workspace constraints. At higher levels of abstraction, robot programming can be categorized into manual programming, offline programming, and task-level programming [3]. Manual programming involves direct interaction with the robot using teach pendants, where operators define positions and movements explicitly. Offline programming (OLP) introduces the use of virtual environments and CAD-based models to simulate and validate robot programs before execution. Task-level programming further abstracts robot control by allowing users to specify high-level goals, which are then decomposed into executable motion sequences automatically [4]. In recent years, research has focused on increasing the level of abstraction in robot programming to reduce dependency on expert knowledge and improve scalability in flexible manufacturing systems. Approaches based on motion primitives, skill-based programming, and knowledge-driven frameworks have been proposed to bridge the gap between high-level task specifications and low-level robot execution [5]. These developments form the basis for modern automated robot programming systems and are particularly relevant in complex

applications such as robotic sheet metal bending, where multiple stages of manipulation, reorientation, and constraint handling are required.

2.2 Sheet Metal Bending and Press Brake systems

Sheet metal bending is a fundamental manufacturing process in which a flat metal sheet is plastically deformed into a desired shape by applying force using a press brake. A typical press brake system consists of a punch and die set, where the sheet is positioned and bent along predefined lines. The bending angle and final geometry depend on factors such as tool geometry, material properties, and applied force [6]. Figure 2.1 illustrates a typical robotic press brake setup, where an industrial robot is integrated with the bending machine to automate handling operations.



Figure 2.1: Robotic press brake bending system with integrated robot handling source -[7]

In automated bending systems, industrial robots are used to perform tasks such as picking the sheet, positioning it within the press brake, reorienting it for subsequent bending operations, and unloading the final part. Robotic integration enables improved repeatability and reduces operator dependency in high-volume production environments [8].

2.2.1 Process Characteristics and Challenges

A key characteristic of sheet metal bending is that the geometry of the workpiece changes after each bending operation. Unlike rigid object manipula-

tion, the robot must continuously adapt to a dynamically evolving part shape. This directly affects, Grasp feasibility, Collision avoidance, Reachability constraints, Orientation planning. Studies have shown that the feasibility of bending operations strongly depends on geometric constraints and tool accessibility, requiring careful coordination between robot motion and bending sequences[9].

Another critical challenge is collision avoidance within constrained environments. The robot must operate in close proximity to the press brake tooling, which limits available workspace and increases the risk of collisions. Path planning methods must therefore consider both robot kinematics and environmental constraints[10]. Subsection Regripping and Workpiece Reorientation One of the most complex aspects of robotic sheet metal bending is the requirement for regripping operations. During multi-stage bending, the workpiece often needs to be repositioned to enable subsequent bends. This is typically achieved by placing the sheet at an intermediate location (e.g., orientation table) and re-grasping it in a new configuration. Regrasp planning has been extensively studied in robotics and is typically formulated as a search problem over feasible grasp configurations and transitions between them [11]. However, in industrial bending applications, regripping is often implemented using predefined strategies, limiting flexibility when handling varying geometries [9]. In industrial bending applications, however, regripping is often implemented using predefined strategies due to process constraints. This limits flexibility and increases engineering effort when handling new geometries. As a result, regripping remains a major bottleneck in achieving fully automated bending systems [8], [9].

2.2.2 Grasping and Vacuum-Based Handling

In many sheet metal applications, vacuum grippers are used due to their ability to handle flat surfaces efficiently. However, grasp stability becomes critical, especially when handling large or flexible sheets. Research on vacuum-based gripping systems highlights the importance of modeling suction forces and ensuring sufficient contact area for stable manipulation[12]. Additionally, data-driven approaches to grasp synthesis have been proposed to improve robustness in uncertain environments [13]. In bending processes, grasp selection must also consider future operations, ensuring that the chosen grasp does not interfere with subsequent bends or tool access. This introduces a coupling between grasp planning and process planning.

2.2.3 Trajectory Planning in Bending Environments

Trajectory planning for robotic bending is significantly more complex than standard pick-and-place tasks due to workspace constraints and dynamic geometry changes. The robot must generate collision-free paths while maintaining feasible kinematic configurations.

Early research on robot-assisted bending highlighted the importance of path planning methods that account for both robot motion and process constraints[13]. These methods must ensure that the robot can safely move between intermediate positions, especially during reorientation and regripping operations. Classical trajectory generation methods provide a foundation for generating smooth and feasible robot motion [10]. However, these approaches often assume static environments and do not fully address the challenges introduced by changing workpiece geometry and constrained bending setups.

2.2.4 Limitations of Existing Industrial Systems

Despite significant progress, existing robotic bending systems exhibit several limitations such as , Heavy reliance on predefined handling strategies,Limited adaptability to varying geometries,High engineering effort for process setup,Lack of integration between grasp planning, regrip planning, and trajectory generation Many industrial systems use conservative motion planning to ensure safety, which reduces efficiency when handling complex parts. Additionally, most approaches treat grasping, planning, and execution as separate stages, leading to suboptimal performance.

2.2.5 Relevance to This Thesis

The challenges identified in robotic sheet metal bending,particularly dynamic geometry handling, regripping, and constrained trajectory planning,directly motivate the approach developed in this thesis. Unlike traditional systems, the proposed implementation integrates, Automated trajectory generation, Constraint-aware motion planning, Structured execution pipelines This enables more flexible and scalable automation of bending operations, addressing key limitations identified in the state of the art.

2.3 Manual and Teach Pendant Programming

Manual robot programming using teach pendants represents one of the earliest and most widely adopted approaches in industrial robotics. In this method, robot motions are explicitly defined by an operator through direct interaction with the robot controller. The operator either guides the robot to desired positions or manually inputs Cartesian or joint coordinates, which are stored as waypoints and executed sequentially by the control system. Figure 2.2 illustrates a typical teach pendant interface used for manual robot programming. This programming approach provides a high level of control

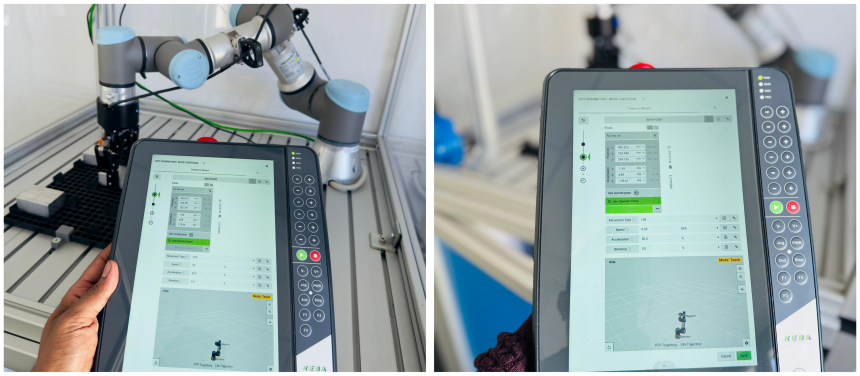


Figure 2.2: Teach pendant interface for manual robot programming (Source: Own work9)

and is particularly suitable for structured and repetitive tasks. It allows operators to directly verify robot motions during programming, reducing the risk of unexpected behavior during execution. As a result, manual programming remains widely used in industrial environments where tasks are stable and do not require frequent modification [3].

2.3.1 Limitations of Manual Programming

Despite its advantages, manual programming exhibits several limitations, particularly in flexible manufacturing scenarios. One major drawback is the high dependency on operator expertise, as programming requires knowledge of robot kinematics, coordinate systems, and process constraints. This increases setup time and introduces variability depending on the skill level of the operator. Another limitation is the lack of scalability. When product geometry changes, the entire sequence of waypoints must often be redefined manually. This makes manual programming inefficient for applications involving high product variability or frequent reconfiguration [13]. Furthermore, manual programming does not inherently account for process con-

straints such as collision avoidance, reachability, or optimal path planning. These factors must be considered explicitly by the operator, increasing the complexity of programming and the likelihood of errors

2.3.2 Challenges in Complex Manipulation Tasks

In applications such as robotic sheet metal bending, manual programming becomes particularly challenging due to the need for multi-stage manipulation and reorientation. The operator must anticipate changes in workpiece geometry and define intermediate waypoints accordingly. Additionally, re-gripping operations require careful coordination of placement and grasping actions, which are difficult to define manually in a consistent and scalable manner. As task complexity increases, the number of required waypoints grows significantly, leading to longer programming times and reduced efficiency. These limitations highlight the inadequacy of manual programming for complex and dynamic industrial processes, where automation and adaptability are required.

2.3.3 Transition Towards Automated Programming

The limitations of manual programming have motivated the development of higher-level programming approaches, including offline programming and automatic program generation. These approaches aim to reduce reliance on operator intervention by generating robot motions based on process requirements and system constraints. In modern industrial practice, there is a clear shift from manual programming towards automated and rule-based approaches, particularly in applications involving variable geometries and multi-stage processes. This transition forms the basis for the approaches discussed in the subsequent sections.

2.4 Offline Programming Approaches for Industrial Robots

Offline programming (OLP) is an established approach in industrial robotics that enables the development and validation of robot programs within a virtual environment prior to execution on physical systems. In contrast to manual programming, OLP decouples programming from production, allowing robot trajectories and task sequences to be generated without interrupting ongoing operations.

Figure 2.3 illustrates a typical offline programming environment, where a virtual model of the robot, tooling, and workcell is used to simulate and

validate motion trajectories.

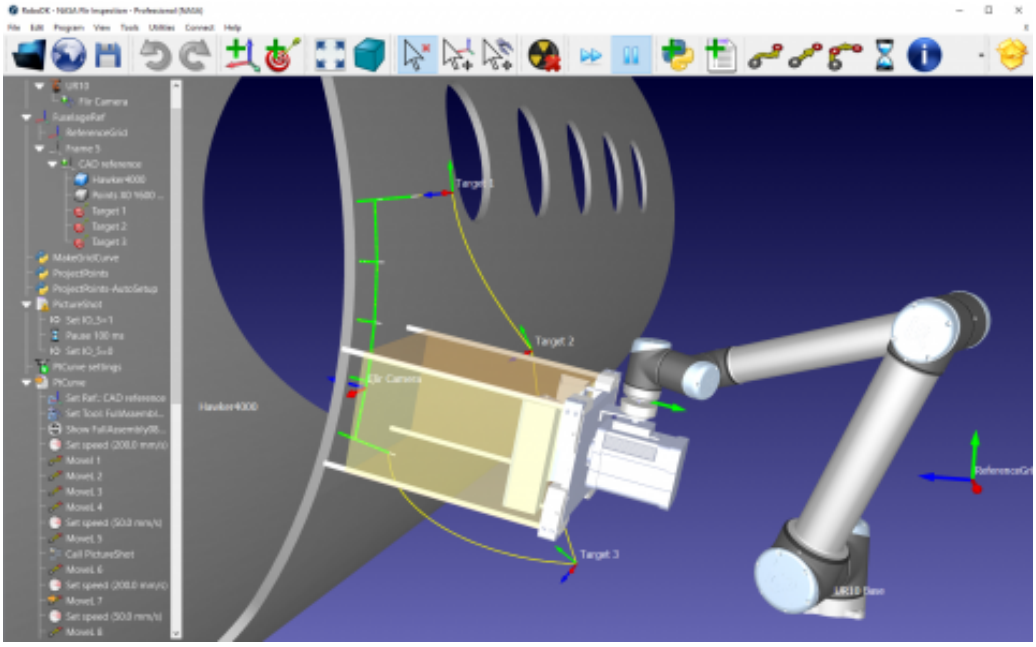


Figure 2.3: Offline robot programming environment with CAD-based simulation (Source: [14])

Offline programming systems typically rely on CAD models and geometric representations of the robot and its environment. These models are used to simulate robot motion, evaluate reachability, and detect potential collisions before execution. This approach significantly reduces downtime and improves safety by allowing errors to be identified during the planning phase [15].

2.4.1 Digital Twin and Simulation-Based Programming

A key development in offline programming is the use of digital twins, which represent a virtual replica of the physical robotic system. Digital twin models enable real-time synchronization between the simulated environment and the physical system, allowing more accurate validation of robot programs. Digital twin-driven manufacturing approaches have been widely studied, demonstrating their effectiveness in improving system reliability and reducing commissioning time [16]. By incorporating sensor data and system feedback, digital twins can enhance the accuracy of simulation-based planning. In industrial practice, simulation platforms such as Drag and bot RoboDK, ABB RobotStudio, and KUKA.Sim provide comprehensive environments for offline programming, including path planning, collision detection, and code generation. Advantages of Offline Programming compared to manual programming, which Reduced production downtime, as programming is

performed outside the physical system, Improved safety due to collision detection and validation in simulation, Enhanced visualization, enabling operators to analyse robot motion before execution, Support for complex environments, where direct manual programming is difficult. These advantages make OLP a widely adopted solution in industrial automation, particularly for applications requiring precise and repeatable motion.

2.4.2 Limitations of Offline Programming

Despite its benefits, offline programming has several limitations. One major challenge is the dependency on model accuracy. Any mismatch between the virtual model and the real environment can result in deviations during execution, requiring manual correction. Furthermore, offline programming still relies heavily on manual input for defining trajectories and task sequences. Although simulation tools assist in validation, they do not inherently generate robot programs automatically. As a result, significant engineering effort is still required to adapt programs to new workpiece geometries or process changes [13]. Another limitation is the lack of integration with higher-level task planning. In many systems, offline programming is used primarily for trajectory validation rather than for fully automated task generation. This limits its ability to handle dynamic and complex industrial processes such as robotic sheet metal bending. While offline programming improves efficiency compared to manual programming, it does not eliminate the need for human intervention in defining robot motions. In applications involving dynamic geometry changes and multi-stage manipulation, such as sheet metal bending, these limitations become significant. The approach developed in this thesis builds upon the advantages of simulation-based environments while addressing their limitations by introducing automated trajectory generation and constraint-aware motion planning. Unlike traditional offline programming, the proposed system generates robot trajectories programmatically and executes them through a structured pipeline, reducing reliance on manual programming.

2.5 Automatic Robot Program Generation and Rule-Based Approaches

Automatic robot programming aims to reduce or eliminate manual intervention by generating robot programs directly from high-level task descriptions, process parameters, or environmental inputs. Unlike manual and offline

programming approaches, which require explicit definition of trajectories, automatic programming systems compute robot motions programmatically based on predefined rules, constraints, or models.

2.5.1 Task-Level Programming and Abstraction

A fundamental concept in automatic robot programming is task-level programming, where users specify what the robot should achieve rather than how it should move. The system then decomposes high-level tasks into sequences of executable actions. Early work on robot programming introduced the concept of abstraction layers, enabling robot programs to be generated from symbolic descriptions of tasks [4]. This approach reduces the need for detailed motion specification and allows systems to operate at a higher level of reasoning. More recent research has focused on skill-based programming, where tasks are represented as reusable motion primitives or skills. These skills encapsulate common operations such as grasping, placing, or approaching, and can be combined to form complex task sequences [5]. This modular approach improves scalability and reusability in industrial applications.

2.5.2 Rule-Based and Constraint-Based Planning

In structured industrial environments, automatic programming is often implemented using rule-based systems, where robot behaviour is defined through a set of logical rules and constraints. These rules govern decisions such as grasp selection, motion sequencing, and collision avoidance. Constraint-based approaches extend this concept by incorporating geometric, kinematic, and process constraints into the planning process. The system evaluates feasible solutions based on these constraints and generates valid robot trajectories accordingly. Such approaches are particularly effective in applications where process conditions are well-defined, enabling deterministic and reliable program generation. However, the complexity of rule sets increases significantly with task variability, making system design and maintenance more challenging [17].

A key challenge in automatic robot programming is the integration of multiple planning components, including Grasp planning, Regrip planning, Trajectory generation, Collision avoidance. In many existing systems, these components are developed independently, resulting in fragmented workflows and limited adaptability. For example, grasp planning methods may not consider future trajectory constraints, leading to infeasible execution

scenarios. Research in robotic manipulation highlights the importance of integrating these components into a unified framework to achieve robust and efficient task execution [18]. However, achieving such integration in industrial environments remains an open challenge due to the complexity of real-world constraints. Despite significant advancements, current automatic programming approaches exhibit several limitations: such as, Limited integration between planning modules, grip position finding for changing workpiece geometry, collsion avoidance, Scalability issues in rule-based systems, Dependency on structured environments, Insufficient adaptability to dynamic geometry changes. These limitations are particularly evident in applications such as robotic sheet metal bending, where the workpiece geometry evolves during the process and requires continuous adaptation of robot actions. The limitations identified in automatic robot programming directly motivate the approach developed in this thesis. While existing methods provide partial automation, they do not fully integrate all required planning components into a unified pipeline. The proposed system addresses this gap by, combining trajectory generation, constraint handling, and execution into a structured workflow, generating robot motions automatically from validated target poses, Reducing reliance on manual programming and predefined rules This integrated approach enables more flexible and scalable automation, particularly in complex applications such as robotic sheet metal bending.

2.6 Robotic Sheet Metal Bending System

Robotic sheet metal bending systems integrate industrial manipulators with press brake machines to automate the handling and positioning of sheet metal components during bending operations. In such systems, the robot is responsible for transporting the workpiece, aligning it accurately within the press brake tooling, and reorienting it between successive bending steps. This integration improves process repeatability, reduces operator dependency, and enhances safety in industrial environments.

Figure 2.4 illustrates a typical robotic bending setup, where the robot operates in coordination with the press brake to execute a sequence of manipulation and forming operations. The bending process typically follows a structured sequence: the robot first picks the flat sheet from an initial location, positions it within the press brake for the first bend, and subsequently performs a series of repositioning actions to enable additional bends. After each bending operation, the geometry of the workpiece changes, requiring

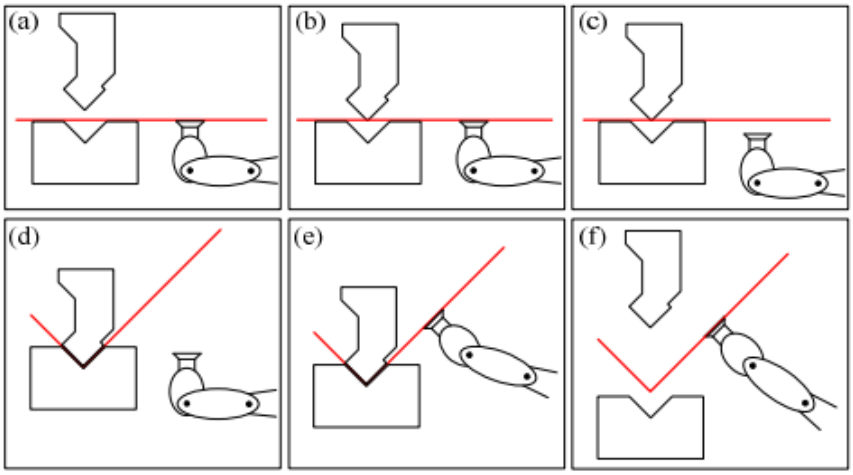


Figure 2.4: Robotic sheet metal bending system with integrated robot and press brake ([19])

the robot to adapt its grasp and motion strategy accordingly. This sequential transformation of the workpiece introduces significant complexity compared to standard pick-and-place operations.

Research on robotic bending systems has demonstrated that successful automation depends on the coordination between process planning and robot motion planning. In particular, the feasibility of bending operations is influenced by geometric constraints, tool accessibility, and robot reachability [9]. These factors must be considered jointly to ensure that each bending step can be executed without collisions or infeasible configurations.

Although robotic bending systems have been successfully deployed in industrial settings, they often rely on predefined handling strategies and carefully engineered workflows. This limits their flexibility when dealing with varying part geometries or dynamic production requirements. As a result, achieving fully automated and adaptive bending systems remains an ongoing research challenge.

2.6.1 Regrip Planning in Sheet Metal Bending

One of the defining challenges in robotic sheet metal bending is the need for regripping, where the robot must change its grasp on the workpiece to enable subsequent bending operations. Since the orientation required for each bend differs, a single grasp configuration is typically insufficient for the entire process.

Regripping is commonly performed by placing the workpiece at an intermediate location—such as a support table or reorientation fixture—and then grasping it again in a different configuration. This introduces additional plan-

ning complexity, as the system must determine both feasible intermediate placements and valid grasp transitions.

From a robotics perspective, regrasp planning is often formulated as a search problem over a set of candidate grasp configurations and transitions between them. Efficient methods must evaluate grasp stability, collision constraints, and reachability to identify feasible sequences [11]. However, the combinatorial nature of the problem makes it computationally challenging, particularly for complex geometries.

In industrial bending applications, regripping is frequently handled using predefined rules or manually designed sequences. While this approach ensures reliability for specific parts, it lacks scalability and requires significant engineering effort when adapting to new geometries. This limitation is particularly relevant in flexible manufacturing environments, where product variation is high.

2.6.2 Challenges of Regrip Integration in Industrial Systems

While regrasp planning has been extensively studied in robotics, its integration into industrial bending systems remains limited. One of the primary challenges is the tight coupling between regripping, process planning, and robot motion. Decisions about where and how to regrip the workpiece directly influence the feasibility of subsequent bending operations and trajectories. In many existing systems, regrip strategies are determined independently of trajectory planning. This separation can lead to situations where a valid grasp configuration results in an infeasible robot motion, or where a feasible trajectory cannot be executed due to grasp constraints. As a result, achieving a consistent and reliable workflow requires careful coordination between these components. Another challenge arises from the changing geometry of the workpiece. After each bend, the shape of the sheet alters, affecting both grasp feasibility and collision conditions. The robot must continuously adapt its manipulation strategy to account for these changes, which significantly increases planning complexity. Furthermore, industrial systems often prioritise robustness and repeatability over flexibility. As a result, many implementations rely on conservative strategies that restrict the solution space to predefined configurations. While this improves reliability, it reduces the system's ability to handle new or unstructured tasks. These challenges highlight the need for integrated approaches that combine regrip planning with trajectory generation and constraint handling. Addressing this integration is essential for achieving scalable and fully automated robotic bending sys-

tems.

2.6.3 Grasp Planning for Sheet Metal Handling

Handling sheet metal components introduces unique challenges in robotic manipulation due to their geometry, size, and material properties. Unlike rigid objects with well-defined grasp points, sheet metal parts are often thin, large, and flexible, making stable grasping non-trivial. In industrial applications, vacuum-based grippers are commonly used for sheet metal handling, as they provide a practical solution for grasping flat surfaces. However, the effectiveness of such systems depends on factors such as suction cup placement, contact area, and force distribution. Improper grasp selection can lead to slippage, deformation, or loss of control during manipulation [12]. From a robotics perspective, grasp planning involves determining suitable contact points and configurations that ensure stability throughout the manipulation process. Data-driven and analytical approaches have been developed to address this problem, considering both object geometry and task constraints [18]. However, in bending applications, grasp planning is further complicated by the requirement to maintain accessibility for subsequent operations. A key challenge is that the selected grasp must remain valid across multiple stages of the bending process. Since the workpiece undergoes geometric transformation after each bend, grasp configurations that are feasible at one stage may become invalid at later stages. This introduces a dependency between grasp planning and process sequencing, requiring coordinated decision-making. In industrial practice, grasp strategies are often predefined based on experience and part geometry. While this approach ensures reliability for known tasks, it limits flexibility and increases setup effort when handling new workpieces. As a result, automated and adaptive grasp planning remains an open challenge in robotic sheet metal bending.

2.6.4 Trajectory Planning in Robotic Bending Systems

Trajectory planning in robotic sheet metal bending systems is significantly more complex than in conventional pick-and-place tasks. The robot must generate motion paths that are not only collision-free but also kinematically feasible within a constrained workspace. One of the primary challenges is the proximity to press brake tooling, which restricts available motion space and increases the risk of collisions. The robot must carefully navigate around the punch and die while maintaining precise positioning of the workpiece. This requires accurate modeling of both the robot and the environment dur-

ing path planning. Additionally, the changing geometry of the workpiece introduces dynamic constraints. After each bending operation, the shape of the sheet changes, altering collision boundaries and reachability conditions. Consequently, trajectory planning must account for these variations at every stage of the process. Classical trajectory generation methods provide a foundation for generating smooth and continuous robot motion [10]. However, these methods typically assume static environments and do not explicitly address the complexities introduced by evolving workpiece geometry and process constraints. Another important consideration is the avoidance of kinematic singularities and workspace limits. Industrial manipulators, such as articulated robot arms, exhibit configurations where small changes in Cartesian motion result in large joint movements. Planning trajectories that avoid such configurations is essential for maintaining stable and predictable robot behavior. In industrial implementations, trajectory planning is often simplified through conservative motion strategies or manually defined waypoints. While this ensures robustness, it reduces efficiency and limits adaptability. More advanced approaches aim to generate trajectories automatically while incorporating constraints such as workspace boundaries, orientation limits, and process-specific requirements.

2.6.5 Limitations of Existing Robotic Bending Systems

Despite advancements in robotic manipulation and process planning, existing robotic sheet metal bending systems still exhibit several significant limitations. A primary limitation is the lack of integration between key planning components, including grasp planning, regrip planning, and trajectory generation. In many systems, these components are treated as separate modules, leading to fragmented workflows and reduced overall efficiency. As a result, decisions made in one stage may negatively impact feasibility in subsequent stages. Another critical issue is the continued dependence on manual programming and predefined strategies. Even when simulation tools and offline programming environments are used, engineers are typically required to define motion sequences, select grasp configurations, and validate trajectories manually. This increases setup time and limits scalability in flexible manufacturing scenarios. Furthermore, existing systems often struggle with adaptability to changing workpiece geometry. Since the shape of the sheet evolves during the bending process, planning approaches must continuously adapt to new constraints. Many industrial solutions address this challenge through conservative assumptions or fixed workflows, which

restrict flexibility and reduce efficiency. Regripping operations also remain a major bottleneck. The lack of automated regrip planning and integration with motion planning leads to increased complexity and reduced robustness. This is particularly problematic in multi-stage bending processes, where multiple reorientation steps are required.

Finally, trajectory planning methods used in industrial systems are often simplified to ensure reliability. While this approach improves robustness, it limits the ability to generate optimal or adaptive motion paths, particularly in constrained environments. In summary,the analysis of robotic sheet metal bending systems highlights that, despite significant progress, several challenges remain unresolved. In particular, the lack of integration between grasp planning, regrip planning, and trajectory generation limits the ability of existing systems to operate efficiently in dynamic and flexible manufacturing environments. These limitations indicate that current approaches are not sufficient to fully automate complex bending processes. Addressing these challenges requires a unified framework that combines constraint-aware planning, automated trajectory generation, and structured execution. This observation forms the basis for the research gap identified in the following section, where the need for an integrated and automated robotic bending system is established.

2.7 Industrial Software Platforms and System Implementations

Industrial robotic applications are commonly supported by specialised software platforms that provide capabilities for robot programming, simulation, and execution. These tools play a central role in bridging the gap between theoretical planning approaches and real-world deployment by enabling engineers to design, validate, and operate robotic systems within structured environments.

2.7.1 Simulation and Offline Programming Environments

Simulation-based tools are widely used in industrial robotics to develop and validate robot programs before deployment, enabling safe and efficient system commissioning [15].These systems allow users to evaluate reachability, perform collision checks, and analyse robot motion within a controlled setting. By enabling offline development, they help reduce production downtime and improve safety during system commissioning. In addition, they support the generation of executable robot code, facilitating the transfer of

validated programs to physical systems. Despite these capabilities, such platforms typically require users to manually define task sequences, select waypoints, and configure motion parameters. As a result, they function primarily as development and validation tools, rather than fully automated planning systems.

2.7.2 Graphical Programming and Integrated Execution Systems

In addition to simulation tools, modern platforms such as drag&bot provide integrated environments that combine graphical programming, simulation, and runtime execution. [20] .These systems enable users to construct robot workflows using modular function blocks, reducing the need for low-level programming. Such platforms often include communication interfaces that connect high-level applications to the robot controller. In practical implementations, this is commonly achieved through middleware or network-based communication mechanisms, allowing external systems to send commands and receive feedback during execution. While these environments significantly improve usability and deployment speed, they still depend on user-defined workflows and manual configuration of task logic. In complex applications, such as robotic sheet metal bending, engineers must still define how tasks are sequenced and how motions are executed. Although industrial robot programming tools have evolved considerably, they exhibit several limitations when applied to complex manipulation tasks. A key limitation is the absence of integrated planning across multiple stages of the process. Grasp selection, regrip handling, and trajectory generation are not automatically coordinated within these platforms, requiring users to manage these aspects manually. Furthermore, many systems rely on predefined workflows and templates, which reduces flexibility when dealing with new or varying workpiece geometries. This makes it challenging to scale solutions in environments where product variation is high. Another important limitation is that these platforms primarily support execution and validation, rather than automated decision-making. While they provide powerful tools for implementing robot programs, they do not inherently generate task strategies or motion plans based on process requirements.

2.7.3 Relevance to This Thesis

The capabilities and limitations of existing industrial platforms highlight an important gap between available tools and the requirements of complex robotic applications. While simulation and execution environments provide

a strong foundation, they do not eliminate the need for manual definition of task workflows and motion strategies.

The work presented in this thesis builds upon these platforms by introducing an automated approach to trajectory generation and execution. By integrating planning and execution within a structured pipeline, the proposed system reduces reliance on manual configuration and enables more flexible handling of multi-stage manipulation tasks. This perspective positions the proposed approach as a complementary layer to existing industrial tools, extending their functionality rather than replacing them.

2.8 Research Gap and Motivation

The review of existing robot programming approaches and robotic sheet metal bending systems reveals a clear progression from manual programming to simulation-based and partially automated methods. Manual programming techniques, while reliable and widely adopted, require significant operator expertise and are not scalable for applications involving frequent changes in workpiece geometry. Offline programming approaches improve efficiency by enabling simulation-based validation; however, they continue to rely on manual definition of task sequences and trajectories. More advanced automatic programming methods introduce higher levels of abstraction, yet they often focus on individual components of the problem rather than providing a fully integrated solution.

In the specific context of robotic sheet metal bending, these limitations become more critical due to the inherent complexity of the process. The geometry of the workpiece evolves after each bending operation, requiring continuous adaptation of grasp configurations, reorientation strategies, and motion planning. While individual solutions for grasp planning, regrip planning, and trajectory generation have been extensively studied, they are typically developed and applied independently. As a result, the interaction between these components is not adequately addressed in existing systems.

This lack of integration leads to several practical challenges. First, decisions made during grasp selection may result in infeasible trajectories or require additional regripping steps. Second, regrip strategies are often defined without considering the constraints of subsequent robot motion, leading to inefficiencies or execution failures. Third, trajectory planning is frequently performed independently of grasp and process constraints, resulting in conservative or suboptimal motion strategies. These issues are particularly pronounced in constrained environments such as press brake cells,

where workspace limitations and collision risks must be carefully managed.

Furthermore, existing industrial implementations continue to rely heavily on predefined workflows and manual intervention to ensure reliable operation. Even when simulation tools and advanced programming platforms are employed, engineers are typically required to define motion sequences, validate feasibility, and adjust parameters manually. This dependence on manual configuration limits scalability and makes it difficult to adapt to new part geometries or changing production requirements.

Based on this analysis, a fundamental research gap can be identified. There is currently no unified and practically deployable approach that simultaneously integrates grasp planning, regrip planning, and constraint-aware trajectory generation within a single automated workflow for robotic sheet metal bending. In particular, existing methods do not adequately address the need to solve these interdependent problems jointly under workspace, kinematic, and process constraints.

Addressing this gap is essential for enabling flexible and scalable automation in industrial bending applications. A system capable of integrating these components can reduce engineering effort, improve adaptability, and enhance overall process efficiency.

To overcome these limitations, this thesis proposes an integrated approach in which validated target poses are transformed into executable robot trajectories through a structured and constraint-aware pipeline. The system combines automated trajectory generation, workspace-aware motion planning, and deterministic execution, thereby reducing reliance on manual programming and predefined strategies. By addressing the interaction between grasping, regripping, and motion generation, the proposed approach contributes towards more robust and scalable robotic bending systems.

The identified gap and the proposed direction form the basis for the concept development presented in the following chapter.

3 System Concept and Methodology

3.1 Problem Definition and Objectives

Industrial robots are widely used in press brake sheet metal bending processes, primarily through manual programming or predefined programming approaches. These methods are effective in structured production scenarios where part geometry, position, and bending sequences remain constant. However, in modern manufacturing environments where product variation is high, these approaches require frequent manual reprogramming, resulting in increased setup time and reduced efficiency .

A key challenge in robotic sheet metal bending arises from the fact that the geometry of the workpiece changes after each bending operation. These geometric transformations directly affect robot reachability, grip feasibility, and the selection of valid grip points, as well as the generation of feasible motion trajectories. Consequently, handling and regripping operations must be carefully planned at each stage of the process. Previous research indicates that managing these aspects is one of the most complex and time-consuming tasks in robotic bending, particularly when performed manually.

From a methodological perspective, conventional robot programming approaches operate at a low level of abstraction. In such approaches, programmers are required to explicitly define robot paths, tool configurations, and individual waypoints. This results in programs that are tightly coupled to specific robot setups, grip positions, and tool configurations, making them difficult to adapt when reorientation or regripping is required. As system complexity increases, these low-level programming approaches do not scale effectively, particularly in flexible manufacturing environments where frequent adjustments are necessary [21].

Although research has explored higher-level programming approaches that integrate motion execution with decision-making in a unified framework, such methods have not been fully applied to press brake sheet metal bending processes where changes in workpiece geometry and position must be explicitly considered. As a result, there remains a lack of a systematic and integrated approach for handling these challenges.

Based on these observations, the central problem addressed in this thesis can be defined as follows: There is a lack of a systematic approach for the automatic generation of robot programs for press brake sheet metal bending

that explicitly considers changing workpiece geometry, handling feasibility, and regripping requirements, while minimizing manual programming effort.

The central objective of this thesis is the automatic generation of robot programs for sheet metal handling and bending operations, thereby reducing manual programming effort and improving adaptability to varying workpiece geometries.

To achieve this objective, this work proposes a conceptual and methodological framework that integrates bending, handling, regripping, and orientation management within a unified workflow. The system is designed to automatically generate robot trajectories and execution logic based on input parameters, without requiring manual definition of waypoints or motion sequences.

The specific objectives of this work are:

- Develop a method for the automatic generation of robot programs based on workpiece geometry and process parameters.
- Generate feasible robot trajectories while considering geometric, kinematic, and workspace constraints.
- Support multi-stage bending operations, including reorientation and regripping of the workpiece.
- Develop a method for determining feasible grip points based on geometry, accessibility, and process stage.
- Enable integration with an industrial robot programming environment through a communication interface.
- Reduce manual programming effort while ensuring consistent and reliable system behaviour.

The proposed approach is validated within a virtual simulation environment compatible with industrial robot systems. The scope of this thesis is limited to conceptual development and simulation-based evaluation.

3.2 Requirement Definition and Management

The development of the proposed system is guided by a structured set of requirements derived from the problem definition presented in Section 3.1. These requirements define the expected functionality, system behaviour,

and operational constraints, and ensure that the solution aligns with the objective of automatic generation of robot programs for sheet metal bending operations.

To maintain clarity and traceability throughout the development process, the requirements are organised into functional, non-functional, system-level, and software-specific categories. This structured classification supports systematic design, implementation, and validation of the proposed approach.

3.2.1 Functional Requirements

The functional requirements define the core capabilities of the proposed system, with a primary focus on enabling the automatic generation of robot programs for sheet metal bending operations. These requirements describe how the system processes input data, generates robot motion, and ensures feasibility across multiple stages of the bending process.

The most critical requirement of the system is the ability to automatically generate robot programs based on operator-defined inputs, thereby eliminating the need for manual programming of robot trajectories and motion sequences. This requirement directly addresses the limitations identified in existing approaches and forms the foundation of the proposed solution.

To achieve this, the system shall generate feasible robot trajectories for bending and handling operations based on predefined target poses and process parameters. The generated trajectories must satisfy geometric and kinematic constraints to ensure that the robot can physically execute the planned motion within the defined workspace.

The system shall support multi-stage bending processes, including reorientation and regripping of the workpiece. Since the geometry of the sheet changes after each bending operation, the system must determine feasible grip points dynamically based on the current process stage, accessibility, and workpiece configuration. This ensures stable handling and continuity of the bending sequence.

In addition, the system shall validate all generated motions prior to execution. If infeasible conditions are detected, such as unreachable poses or invalid configurations, the system shall prevent execution and initiate a replanning process to generate an alternative feasible solution. If recovery is not possible, the system shall safely terminate execution to avoid unsafe conditions.

Furthermore, the system shall enable simulation-based validation of the generated robot programs within a virtual environment. This allows verifi-

cation of the planned sequence before execution and ensures consistency between the generated program and the system constraints.

The detailed functional requirements corresponding to these capabilities are summarised in Table 3.1.

Table 3.1: Functional Requirements for Automatic Robot Program Generation

ID	Functional Requirement (SMART)	Acceptance Criteria	Verification Method
FR-00	Automatic robot program generation shall be implemented for press brake operations.	The system shall generate robot programs automatically for press brake operations.	Simulation based
FR-01	The system shall detect unreachable target poses before robot motion execution.	The robot shall not execute motion to unreachable poses and shall provide an error message.	Simulation based
FR-02	The system shall allow operator-side selection prior to program execution.	The selected bend side shall be applied correctly to the generated robot program.	Simulation based
FR-03	The system shall execute a regripping sequence when a target pose is unreachable within the current bend side.	Regripping shall be performed successfully and execution shall continue without interruption.	Simulation based
FR-04	The system shall handle workpiece orientation changes during program execution.	Orientation changes shall be applied correctly without violating kinematic limits.	Simulation based
FR-05	The system shall support opposite-side bending operations when required.	The system shall correctly execute opposite-side bending operations.	Simulation based
FR-06	The system shall determine and apply suitable grip positions for each workpiece state.	Grip positions shall be determined and applied correctly for each workpiece state.	Simulation based
FR-07	Detect unsafe conditions and perform replanning or terminate execution	System replans sucessfully with warning	Simulation based

3.2.2 Non-functional Requirements

The non-functional requirements define the quality attributes of the proposed system and ensure that it operates reliably, efficiently, and in a manner suitable for industrial applications. These requirements describe how the system performs under various conditions and how it supports usability, maintainability, and robustness.

The system is designed to provide a structured and user-friendly interaction, enabling operators to define process inputs without requiring detailed knowledge of low-level robot programming. This improves accessibility and reduces the complexity associated with traditional programming approaches.

In addition, the system shall ensure reliable and consistent behaviour

by producing deterministic outputs for identical input conditions. This is particularly important in industrial environments, where repeatability and predictability are essential for safe and efficient operation.

From a design perspective, the system follows a modular architecture, allowing individual components such as planning, validation, and execution to be developed, maintained, and extended independently. This modularity enhances maintainability and enables future improvements without affecting the overall system structure.

The system also incorporates robustness and tolerance to minor deviations in input parameters and process conditions. It is designed to handle variations in workpiece geometry within the defined constraints while maintaining stable operation and consistent output.

Furthermore, the system supports scalability by enabling adaptation to different process configurations and workpiece variations within the predefined scope. This ensures that the approach remains applicable across a range of similar industrial scenarios.

The detailed non-functional requirements corresponding to these quality attributes are summarised in Table 3.2.

3.2.3 System Constraints and Assumptions

The proposed system is developed under a set of clearly defined constraints and assumptions that establish the scope and ensure predictable system behaviour. The geometric modelling of the workpiece is limited to rectangular sheet metal components consisting of exactly four edges aligned with the global coordinate system. Rotated or non-axis-aligned sheet configurations are not considered. A single global wall height parameter is assumed for all bending operations, and variations in wall height across different edges are not supported. In addition, the representation of internal features is restricted to circular holes defined by their centre coordinates and radius. More complex geometries, such as polygonal cut-outs or irregular shapes, are not included in the current implementation.

The bending process is also subject to strict simplifications. Only right-angle (90°) bending operations are supported, and no provision is made for variable or user-defined bend angles. Each edge of the sheet can be associated with only one bending operation, resulting in a one-to-one mapping between edges and bends. This restriction ensures a deterministic and structured bending sequence. Furthermore, the number of bending operations is limited to a maximum of four on each side of the workpiece, including both

Table 3.2: Non-Functional Requirements — NFR-01 to NFR-04 with acceptance criteria and verification methods

ID	Category	Non-Functional Requirement	Acceptance Criteria	Verification Method
NFR-01	Usability	The system shall minimise operator interaction during normal execution. Configuration steps prior to execution shall be limited to essential parameters. The system shall provide meaningful warning and error messages that do not require expert robot programming knowledge.	Operator interaction limited to bend side selection and execution confirmation. Warning messages displayed for all detected unsafe conditions.	Usability validation
NFR-02	Maintainability	The system shall allow process parameters, handling strategies, and system settings to be modified through configuration mechanisms that are fully separated from the core program logic, with no source code changes required. Sufficient terminal logging shall be provided to support systematic debugging, monitoring, and future extension of the system.	New workpiece geometry configurable via YAML file update only, without source code modification. Configuration change validated without rebuild.	Configuration validation test
NFR-03	Robustness	The system shall exhibit robust behaviour under repeated execution cycles. Small variations in workpiece orientation or timing shall not lead to system failure or unsafe behaviour. The system shall handle deviations in a controlled manner.	System completes at least five consecutive bending cycles without failure under nominal variation conditions.	Repeated run validation test
NFR-04	Robustness	The system shall tolerate moderate pose deviations within defined limits without triggering an error or aborting execution. Deviations exceeding the threshold shall trigger a warning and controlled stop.	Pose deviations within the defined tolerance threshold do not cause execution termination. Deviations exceeding the threshold trigger a warning and controlled stop.	Perturbation validation test

primary and opposite-side bending stages.

From a system perspective, the robot configuration, tooling setup, and workspace layout are assumed to remain fixed throughout operation. The motion generation process relies on a predefined tool centre point configuration, and all generated trajectories are expressed with respect to this reference. The system assumes the use of a vacuum-based gripping mechanism controlled through a predefined input-output configuration. Changes to tool geometry, tool centre point definition, or input-output wiring are not dynamically handled and require manual reconfiguration of the system.

The implementation also assumes a structured and predefined representation of objects within the execution environment. The system depends on consistent identification and positioning of workpieces and support structures, and deviations from the expected configuration may lead to incorrect execution behaviour. As a result, the workflow is tightly coupled to the predefined virtual cell setup.

With respect to process capabilities, the system supports only vacuum-based gripping strategies. Grip feasibility is evaluated using simplified physical considerations such as pressure distribution, friction, and load capacity. Alternative gripping methods, including mechanical, magnetic, or multi-fingered grippers, are not considered within the scope of this work.

Finally, the scope of the proposed system is limited to rule-based program generation, geometric reasoning, and simulation-based validation. Advanced functionalities such as dynamic collision-free trajectory generation, real-time motion adaptation, and environment-aware path optimisation are explicitly excluded. These constraints ensure that the system remains deterministic, reproducible, and aligned with the intended focus on automatic program generation within a controlled industrial setting.

3.2.4 Software Platform and Tool requirement

The software platform and tool requirements define the technological environment, communication framework, and execution context necessary for implementing the proposed system. These requirements ensure that the system supports automated generation, validation, and execution of robot programs within a structured and industrially relevant setup.

The software-related requirements are defined as SR-01 to SR-03 and are further detailed in the following subsections.

The system shall operate within a virtual robot programming environment that provides capabilities for simulation, execution, and validation of robot programs. In this work, the drag&bot platform is used as the designated virtual environment, enabling structured testing and verification of generated robot trajectories under controlled conditions. This environment ensures consistency between the planning module and the execution context while supporting simulation-based validation of the generated programs.

The system shall communicate with the execution environment through a unified communication interface that enables the exchange of data between system components. This communication is realised using the AnyRobot interface, which provides an abstraction layer for interacting with the robot

programming environment. The communication mechanism is based on a WebSocket protocol, allowing bidirectional data exchange for transmitting robot target poses, execution commands, and system feedback. This requirement ensures synchronisation between the planning module and the execution environment during program generation and validation.

Furthermore, the system shall ensure that all generated robot motion targets are defined with respect to a predefined tool centre point (TCP) configuration. The TCP serves as the reference frame for all trajectory computations and motion commands, ensuring consistency between the generated robot program and the configuration of the robot within the execution environment. This requirement guarantees that the generated motions are physically meaningful and compatible with the defined tool setup.

In addition, the system shall support structured data exchange between internal components, including input parameters, geometric representations, generated trajectories, and validation results. The software architecture shall follow a modular design, enabling clear separation between planning, validation, and execution modules. This modularity enhances maintainability, scalability, and future extensibility of the system.

Finally, the system shall enable simulation-based validation of generated robot programs within the defined virtual environment prior to execution. This ensures that the generated trajectories comply with system constraints and process requirements, thereby improving reliability and reducing the need for manual intervention.

3.2.4.1 SR-01 Virtual Execution Environment

The system shall operate within a virtual robot programming environment that supports simulation, execution, and validation of robot programs. In this work, the drag&bot platform is used as the designated runtime environment for executing and validating all generated robot trajectories.

The drag&bot environment provides a unified interface for robot motion execution, object handling, and input-output operations, allowing the planning system to interact with the robot model in a controlled and structured manner. This ensures that all generated programs can be evaluated consistently within a virtual representation of the target press brake cell without requiring physical hardware access.

By operating within this environment, the system enables reliable simulation-based validation of robot programs, ensuring that functional requirements related to trajectory generation, handling, and process se-

quencing are verified prior to execution. The use of a virtual runtime environment also supports reproducibility of results and reduces dependency on physical system availability.

Acceptance Criteria — SR-01: The system shall establish a stable connection to the virtual runtime environment, and all generated robot motions, service calls, and input-output operations shall be executed and validated successfully within the simulation environment.

3.2.4.2 SR-02 Communication Interface Requirement

The system shall communicate with the robot execution environment through a unified communication interface that enables structured interaction between the planning module and the runtime system. In this work, the AnyRobot interface is used as the exclusive communication layer between the planning system and the execution environment.

The communication shall be implemented using a WebSocket-based protocol, enabling bidirectional data exchange for transmitting robot motion commands, input-output signals, object handling operations, and system feedback. This approach provides a consistent abstraction layer for interacting with the robot programming environment without requiring direct access to robot-specific control interfaces.

By using a unified communication interface, the system ensures that all commands and execution requests are transmitted in a structured and controlled manner, supporting maintainability and compatibility across different system configurations. This abstraction also enables the planning system to remain independent of specific robot controller implementations.

Acceptance Criteria — SR-02: All communication between the planning module and the execution environment shall occur through the defined communication interface. A valid connection shall be established before command transmission, and all motion, input-output, and service operations shall be successfully executed and acknowledged within the simulation environment.

3.2.4.3 Tool Centre Point (TCP) Definition Requirement - SR-03

The system shall define all robot motion targets with respect to a predefined tool centre point (TCP) configuration. The TCP represents the reference

frame of the end-effector and is used as the basis for all trajectory generation and motion planning operations.

All generated robot poses, including approach positions, handling configurations, and placement targets, shall be expressed in relation to the TCP. This ensures consistency between the planning module and the execution environment, and guarantees that the generated trajectories correspond to the physical geometry of the tool.

The TCP configuration is defined externally and remains fixed during system operation. Any changes to the tool configuration or TCP definition require corresponding updates to ensure alignment between the generated trajectories and the execution environment. This requirement ensures that motion accuracy and handling feasibility are maintained throughout the process.

Acceptance Criteria — SR-03: All generated robot motion targets shall be consistent with the predefined TCP configuration, and any modification to the TCP definition shall be reflected correctly in the generated trajectories without requiring structural changes to the system logic.

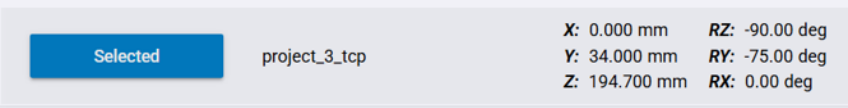


Figure 3.1: Tool Center Point parameters in Drag&Bot.

3.2.5 System Traceability

Requirements traceability is a fundamental principle in systematic engineering development, ensuring that all defined requirements are consistently reflected in the system design, implementation, and validation stages. In the context of the proposed automatic robot program generation system, traceability ensures that the transition from conceptual objectives to executable functionality remains transparent, structured, and verifiable.

Each requirement defined in Section 3.2.1 (Functional Requirements) and Section 3.2.2 (Non-Functional Requirements), as well as the software requirements (SR-01 to SR-03), is assigned a unique identifier (e.g., FR-01, NFR-01, SR-01). These identifiers serve as persistent references throughout the entire development lifecycle and enable systematic tracking of requirements across different stages of the system.

The requirement identifiers are used consistently to establish clear relationships between requirements, system components, and validation procedures. In particular:

- Each requirement is implemented within a specific system module.
- Each requirement is verified through a defined test scenario.
- No requirement remains unaddressed in the implementation.
- All implemented functionalities can be traced back to a corresponding requirement.

Forward Traceability: Forward traceability ensures that every defined requirement is realised in a corresponding system component or functional module. This confirms that all requirements are properly implemented within the system architecture.

Backward Traceability: Backward traceability ensures that every implemented functionality can be traced back to an explicitly defined requirement. This guarantees that no unnecessary or unjustified functionality is introduced into the system.

By mapping each requirement to corresponding implementation modules and validation scenarios, the development process becomes structured, auditable, and reproducible. This is particularly important in systems involving automated decision-making processes, such as reachability evaluation, grip point determination, regripping logic, and opposite-side handling.

In addition, traceability improves system maintainability and supports future extensions. Changes to system requirements can be systematically analysed by identifying the affected components and validation procedures. This enables efficient impact analysis and reduces development risk.

The complete mapping between requirements, system modules, and validation scenarios is presented in Table 3.3.

Table 3.3: Requirement Traceability Matrix — all requirements (FR-00 to FR-07, NFR-01 to NFR-05, SR-01 to SR-03) mapped to implementation modules and verification scenarios

Req. ID	Requirement Description	Implemented In	Verified In
FR-00	Automatic robot program generation for press brake operations.	Integrated planning pipeline combining geometry processing, trajectory generation, and execution logic	End-to-end simulation validation
FR-01	Detect unreachable target poses before execution.	Reachability checking and admissibility validation logic	Simulation test case TC-01
FR-02	Allow operator-side selection of bending configuration before execution.	Operator dialog and configuration collection logic	Execution test case TC-02
FR-03	Execute regripping sequence when a target pose is unreachable.	Automatic regrip cycle logic with I/O and attach/detach control	Regrip validation test case TC-03
FR-04	Handle workpiece orientation changes during program execution.	Orientation conversion, rotation normalisation, and strategy selection logic	Orientation validation test case TC-04
FR-05	Support opposite-side bending operations when required.	Opposite-side bending workflow with sheet-flip and regrip completion logic	Opposite-side validation test case TC-05
FR-06	Determine and apply suitable grip positions for each workpiece state.	Grip planning logic with edge-clearance, spacing, and vacuum-force feasibility checks	Grip validation test case TC-06
FR-07	Detect unsafe conditions and perform replanning or terminate execution.	Safety validation and pre-execution checking logic with replanning mechanism and controlled stop handling	Safety validation test case TC-07
NFR-01	Ensure minimal operator interaction during normal execution.	Operator interaction limited to bend-side selection and execution confirmation	Usability validation
NFR-02	Allow process parameter changes without source code modification.	Configuration reader and YAML-based single-source process configuration	Configuration validation test
NFR-03	Provide clear warning and error messages for unsafe conditions.	Warning generation and modal operator notification logic	Warning message validation
NFR-04	Ensure robustness under repeated execution cycles.	Deterministic execution loop with automatic waypoint generation and path logging	Repeated run validation test
NFR-05	Tolerate moderate pose deviations within defined tolerance limits.	Configurable pose-deviation threshold and perturbation handling logic	Perturbation validation test
SR-01	drag&bot platform shall be the virtual execution environment for robot program validation.	Runtime client connection logic for communication with the configured drag&bot environment	Platform integration check: runtime connection and command execution verified
SR-02	AnyRobot interface shall be the communication layer between planning system and robot runtime.	WebSocket-based communication layer enabling structured command exchange	Interface integration check: connection established and commands successfully executed
SR-03	All motion targets shall be defined with respect to the configured TCP.	TCP-based transformation applied in all trajectory generation and motion execution steps	TCP configuration check: consistent pose transformation verified

4 Development of Concepts

4.1 Purpose and scope of concept development

Concept development is a fundamental stage in systematic engineering design, in which alternative solution principles are generated and evaluated prior to implementation. By comparing different approaches against defined criteria, the risk of selecting an unsuitable design solution is reduced [6].

The research gap identified in Section 2.8 highlights the absence of a systematic approach for automatic robot program generation in press brake sheet metal bending. In particular, existing methods do not adequately address changes in workpiece geometry, gripping, regripping, and pre-execution validation while minimizing manual programming effort. These challenges are translated into measurable engineering objectives and requirements in Chapter 3, including functional requirements (FR-01 to FR-07), non-functional requirements (NFR-01 to NFR-05), and software platform requirements (SR-01 to SR-03), as summarised in Table 3.3.

To address this gap, three conceptually distinct solution approaches are developed and evaluated in this chapter. Concept A represents a structured manual programming workflow, Concept B proposes a rule-based automatic program generation approach, and Concept C explores a CAD-integrated offline programming methodology [3], [21]. These concepts are formulated based on different automation paradigms and are evaluated to identify the most suitable approach for the defined problem.

The selection of the final concept is performed using the Weighted Sum Model (WSM), a quantitative decision-making method in which evaluation criteria are assigned relative weights and each concept is scored accordingly. This ensures that the concept selection process is systematic, transparent, and aligned with the defined requirements.

4.2 Concept A – Structured operator Workflow with Guided Teach in programming

4.2.1 Methodological Framework

Concept A follows a structured manual programming approach in which the operator defines robot motions and process steps explicitly, supported by a

guided workflow. In this approach, robot trajectories, gripper actions, and execution sequences are created manually by the user, while the system provides a structured interface to organise the programming process.

This concept is based on conventional manual robot programming practices, as discussed in Section 2.3, where the operator directly defines waypoints and process logic [3]. The introduction of a guided workflow improves usability by structuring the sequence of operations and reducing unstructured interaction, but it does not change the underlying requirement for manual definition of all robot motions.

Concept A may be implemented within the drag&bot environment using manually configured function blocks for robot motion, gripper control, and execution sequencing. In this case, the operator constructs the workflow by placing and configuring the required functional blocks, including motion commands and input-output operations. However, all program logic, including waypoint definition, gripping strategy, regripping operations, and process sequencing, remains user-defined. As shown in Figure 4.1, the operator manually constructs the robot program by arranging function blocks for motion commands, gripper control, and execution logic within the drag&bot enviornment.

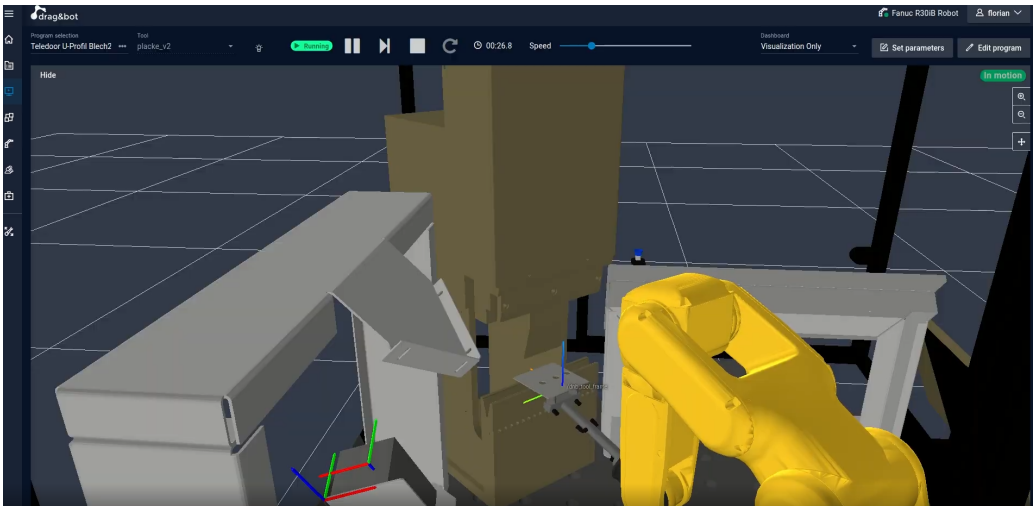


Figure 4.1: Manual robot program construction in the drag&bot environment using function blocks for motion and gripper control.

As a result, the concept does not provide automatic waypoint generation, grip point calculation, regrip planning, or adaptive decision-making based on workpiece geometry. The role of the software environment is limited to manual program construction and execution support, rather than automated program generation.

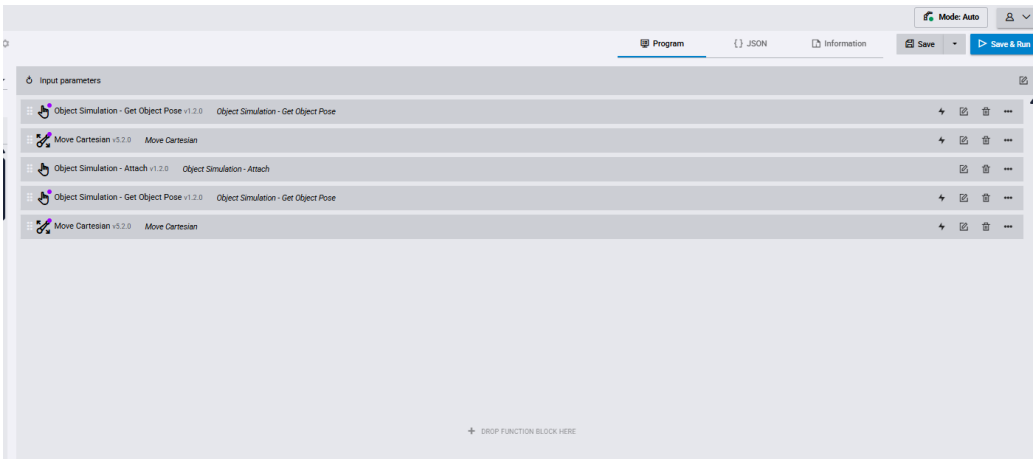


Figure 4.2: Manual robot program construction in the drag&bot environment using function blocks for motion and gripper control.

4.2.2 System Structure

The system structure of Concept A consists of two main layers: a manual workflow construction layer and an execution layer. In the first layer, the operator defines the complete process sequence manually by specifying robot motions, gripper actions, and task order using function blocks or equivalent programming elements within the robot programming environment.

In the second layer, the execution environment interprets and executes the manually defined sequence. No intermediate planning or decision-making layer exists for automatic trajectory generation, geometric reasoning, or constraint evaluation. As a result, the system relies entirely on operator-defined logic and does not incorporate automated validation or adaptive behaviour prior to execution.

4.2.3 Critical Evaluation

CConcept A is straightforward to implement and can be applied within existing robot programming environments, including drag&bot. Its operation is transparent and fully controlled by the operator, allowing direct modification of robot motions and process sequences. These characteristics make it suitable for simple and repetitive production scenarios with limited variability.

However, the limitations of this concept are significant in the context of the research objectives. The programming effort remains high, as all robot motions, handling steps, gripper actions, and process logic must be defined manually for each task. This requires substantial operator expertise in robot kinematics, tooling constraints, and process sequencing [3], [21]. As a result, the concept does not reduce manual programming effort and therefore

fails to address the primary objective of this thesis.

Furthermore, the approach lacks scalability and adaptability. Changes in workpiece geometry, process sequence, or setup conditions require manual modification of the program, which limits its applicability in flexible manufacturing environments. In addition, Concept A does not provide automatic grip point determination, regripping strategies, or configuration-driven trajectory generation. Operations such as opposite-side bending and recovery from infeasible configurations remain entirely dependent on operator intervention.

Another limitation is the absence of integrated validation and decision-making. The system does not evaluate geometric feasibility, reachability, or handling constraints prior to execution. As a result, infeasible configurations are typically identified only during execution, which may lead to operational inefficiencies and potential safety risks.

Although Concept A can utilise the drag&bot environment as an execution platform, it remains a manually programmed solution and does not support automatic program generation, adaptive planning, or rule-based decision-making. Therefore, it does not satisfy the functional and non-functional requirements defined in Chapter 3, particularly those related to automation, adaptability, and validation.

For these reasons, Concept A is not suitable as the final solution for addressing the identified research gap.

4.3 Concept B – Automatic Program Generation based on rules with Integrated Constraint Validation.

4.3.1 Methodological Framework

Unlike Concept A, which relies on manual waypoint definition, Concept B eliminates manual programming entirely by introducing a rule-based automatic program generation approach. Concept B differs fundamentally from CAD-based offline programming approaches by generating robot programs dynamically from configuration parameters and runtime validation, rather than relying on pre-defined digital models of the robot cell.

In this approach, all robot waypoints, grip positions, regripping sequences, and opposite-side handling strategies are derived automatically from geometric input parameters and a structured configuration file. Operator interaction is limited to selecting process options and confirming execution, while all motion generation and decision-making processes are

handled by the system. This significantly reduces manual effort and directly addresses the primary objective of automatic program generation defined in Chapter 3.This approach directly satisfies FR-00 by enabling automatic generation of robot programs without manual intervention.

The system communicates with the robot execution environment through the AnyRobot interface provided within the dra&bot platform, using a WebSocket-based communication mechanism [20]. This architecture establishes a clear separation between the planning logic and execution environment, which is consistent with modular automation design principles [22]. As a result, the system operates independently of specific robot controller implementations while maintaining compatibility with the defined execution platform.

A key component of the methodology is the evaluation of workpiece orientation and feasibility prior to motion generation. The system retrieves workpiece pose information and evaluates admissible orientations using a structured multi-stage validation strategy. This ensures that valid handling configurations are identified regardless of the initial placement of the workpiece. Additionally, workspace constraints and admissible zones are defined through configuration parameters, allowing adaptation to different setups without modifying the underlying logic. This approach satisfies the adaptability requirement defined in NFR-02.

Grip planning is performed using a parameterised approach based on workpiece geometry, suction cup properties, and physical constraints. Candidate grip positions are evaluated against edge clearance, spacing constraints, and vacuum holding force requirements [12]. Only physically feasible and stable grip configurations are considered for further processing, ensuring safe handling conditions prior to motion planning.

The system detects unsafe or infeasible conditions prior to execution and initiates replanning where possible, or terminates execution if recovery is not feasible, ensuring safe and reliable operation

When a target pose cannot be reached with the current grip configuration, the system automatically initiates a regripping sequence. This sequence involves repositioning the workpiece and re-establishing a new grip configuration that restores reachability. This automated handling of regripping directly satisfies functional requirement FR-03.

The methodology also supports opposite-side bending operations through a dedicated workflow. After completing primary bending operations, the system evaluates whether opposite-side processing is required and executes a structured sheet reorientation and handling sequence. All validation steps,

including orientation checking, grip feasibility, and reachability assessment, are consistently applied throughout the process [9].

Robot trajectories are generated using linear Cartesian motion commands, ensuring smooth and predictable motion execution. The use of smooth motion profiles reduces dynamic loading on the workpiece and supports stable vacuum gripping [12]. In addition, kinematic constraints such as singularity avoidance are considered during motion generation to ensure feasible and safe robot operation [3].

All motion targets are defined relative to a predefined tool centre point (TCP), ensuring consistency between the planning module and execution environment. This requirement directly satisfies SR-03 and guarantees that generated trajectories are aligned with the physical configuration of the robot tool.

Overall, Concept B represents a configuration-driven, rule-based methodology that enables automatic program generation, integrated validation, and adaptive handling of changing workpiece conditions. The system operates deterministically based on defined input parameters, ensuring repeatability and reproducibility of generated robot programs.

4.3.2 system structure

The structure of Concept B consists of Five layers as defined in section System-Architecture-and-Design. The input and configuration layer that collects all required inputs for the programming. The geometry and constraints evaluation layer applies grip feasibility rules, edge clearance checks and zone-based orientation constraints. The planning and decision logic layer determines the execution strategy, which decides the program to be direct execution or regrip required execution or workpiece flipping strategy, based on reachability and orientation validity . The trajectory planning layer produces linear cartesian waypoints with minimum jerk velocity profiles. The safety and validation layer applies pre-execution pose check before the command is published to the robot runtime.

This WebSocket channel of the Any robot interface allows to connect all five layer to drag and bot virtual simulation run time [20] .The configuration-driven approach allows all process parameters to be trimmed from program logic, allowing the solution to tackle new workpiece geometries simply by modifying the content of configuration files. This was in direct satisfaction of non-functional requirement NFR-02 as described in Section 3.2.2.

4.3.3 Critical evaluation

Concept B is aligned with the research gap stated in Section 2.8 for every dimension identified in the literature. Define waypoints, plan regrips, and sequence work on the opposite side automatically from geometric parameters instead. A pre-execution pose validator guards against unsafe execution states before motion starts. This allows for configuration driven architecture that can adapt to new part geometries without the need for source code changes,satisfying FR-01, NFR-03, NFR-04, and NFR-05. Configuration-driven parameterization satisfies NFR-02. The system operates exclusively within the drag and bot runtime via the AnyRobot Interface, satisfying SR-01 and SR-02. All motion targets reference the pre-defined tool center point, satisfying SR-03. All fifteen requirements in Chapter 3 are therefore addressed by the concept design.

The primary constraints that Concept B operates within are those identified within Section 3.2.3. The system operates within a rectangular sheet geometry with fixed 90-degree bend angle constraints. The integration with the KEBA robot controller hardware, which is part of the KEBA robot driver layer within the drag and bot software stack [20], is identified within the future work section and is not included within the scope of this thesis. All validation has been conducted within the drag and bot virtual simulation environment, which is discussed in this section. The virtual simulation environment used for this evaluation is illustrated in figure 4.3 .The environment represents a simulated press brake cell configured with the drag and bot platform , including the robot , workpiece and interactions required for the program execution .These constraints are explicit and not indicative of design flaws but, rather, define the scope within which the design concept operates.

4.4 Concept C – CAD Integrated Offline Programming with Digital Twin Simulation

4.4.1 Methodological Framework

Unlike Concept B, which performs rule-based program generation dynamically during execution, Concept C relies on a complete CAD-based digital model of the robot cell to generate and validate robot programs offline prior to execution.

Concept C follows a computer-aided design (CAD) integrated offline programming (OLP) approach, in which a digital representation of the robot,



Figure 4.3: Virtual robotic environment used to evaluate the proposed concept (Drag & Bot platform).

tooling, and workpiece is used to plan and simulate robot operations. In this methodology, robot programs are generated within a simulation environment based on the geometric model of the workpiece and the digital representation of the production cell. All motion planning, collision checking, and feasibility evaluation are performed prior to execution, without requiring direct interaction with the physical robot.

The OLP system evaluates bending sequences, grip positions, regripping strategies, and kinematic feasibility using the CAD model of the workpiece and environment. The generated robot program is validated within the simulation environment and then transferred to the robot controller for execution. This approach corresponds to established offline programming methods used in industrial robotics [21].

Industrial OLP tools such as ABB RobotStudio, KUKA.Sim, and FANUC ROBOGUIDE implement similar methodologies and are widely used in structured production environments [21]. The use of digital twin-based simulation enables early detection of unreachable poses, collisions, and process inconsistencies before actual execution, thereby reducing operational risks [23]

The key conceptual difference between Concept B and Concept C lies in how robot programs are generated and validated. Concept B follows a configuration-driven and rule-based approach, where robot trajectories and handling strategies are generated dynamically using input parameters and validated at runtime. In contrast, Concept C follows a model-driven approach, where all planning and validation are performed within a CAD-based simulation environment before execution. As a result, Concept B offers higher flexibility and adaptability to changing workpiece configurations,

whereas Concept C depends on accurate digital models and requires significant preparation effort for each new scenario.

4.4.2 System Structure

This concept consist of four layers .A cad modelling layer provides digital representations of the robot , press brake , tools and workpiece .An OLP and simulation layer uses the URDF(Unified Robot Description Format) representations to plan the algorithm for robot program through built in kinematic solver .A post processing layer converts the generated programs into controller specific syntax's robot controller layer communicates the program for exception. Accurate digital twin representation of all cell components are required at the modelling layer small errors in modelling the digital twin model propagate as execution error that require manual re programming during commissioning.[3] [21].

4.4.3 Critical Evaluation

Concept C offers meaningful automation advantages for structured, high-volume production environments where accurate cell models are available and OLP tool investment is justified. Simulation-based validation provides strong pre-execution assurance, and the approach is supported by mature commercial software [3][23]. These characteristics make it the dominant paradigm in industrial robotic bending automation at the time of writing.

For the scope of this thesis, Concept C presents three fundamental disadvantages that result in its rejection. First, the implementation requires accurate CAD models of the complete robot cells, including the robot, press brake, tooling, fixtures, and workpieces — which represents a substantial engineering preparation effort that is not available within the thesis timeframe.

Second, commercial OLP systems involve high license fees while demanding expert-level instruction - resources simply unavailable within this project's constraints REF Engineering Design: A Systematic Approach . Besides cost, mainstream OLP platforms lack native compatibility with both the drag and bot simulation framework and the AnyRobot WebSocket system, which define the core setup targeted here [20]. Connecting such a tool would mean building a bespoke bridge to the drag and bot engine - an extra burden unjustified by timeline or ambition.. Even if the first two limitations were resolved, this third limitation would disqualify Concept C,

because SR-01 and SR-02 are non-negotiable constraints from the industrial partner environment.

4.5 Structural Comparison of Concepts

Table 4.1: Structural comparison of Concept A, Concept B, and Concept C across key design features

Design Feature	Concept A	Concept B	Concept C
Waypoint Generation Method	Manually defined by the operator for each task using teach-in or function blocks; no automation [3]	Automatically generated from sheet geometry and configuration parameters; no manual waypoint definition required [20]	Generated using CAD-based offline programming; requires model setup for each geometry variant [21]
Regrip Handling	No automation; regripping is manually planned and executed by the operator [8]	Automated through dedicated regrip logic including IO control, attach/detach operations, and sequence management [8], [9]	Regrip positions defined in simulation; execution remains manual or pre-defined without runtime adaptation [23]
Opposite-Side Bending	Manual sheet flipping and repositioning; no system support	Structured flip and handling sequence implemented through predefined logic and validated execution workflow [9]	Simulated in CAD environment; execution logic not dynamically adapted during runtime [23]
Pre-Execution Validation	Not supported; infeasible poses identified only during execution [3]	Integrated validation of orientation, reachability, and feasibility prior to execution, with support for replanning or safe termination	Validation performed in simulation environment; no runtime validation during execution [21]
Adaptability to Geometry Changes	Requires manual reprogramming for each new workpiece; low adaptability [21]	Configuration-driven approach allows parameter updates without modifying program logic	Requires modification of CAD models and re-simulation for each geometry variation [23]
Platform Compatibility	Can be executed within drag&bot as a manually constructed workflow, but does not utilise the AnyRobot Interface and therefore does not satisfy SR-02	Fully compatible with drag&bot runtime and AnyRobot Interface as defined in SR-01 and SR-02 [20]	Not compatible with drag&bot runtime or AnyRobot Interface; does not satisfy SR-01 and SR-02
Tool Centre Point (TCP) Handling	Defined manually by the operator; not enforced through system-level configuration	All motion targets defined relative to a predefined TCP, ensuring consistency with system configuration (SR-03)	Defined within CAD model; not aligned with system-specific TCP requirements
Implementation Feasibility	Easy to implement but provides no automation capability	Fully implementable within the defined software environment and thesis scope	Requires commercial OLP tools and detailed digital models; exceeds practical scope [3]
Research Gap Coverage	Does not address automation; relies on manual programming	Directly addresses the research gap through automatic program generation, regripping, and validation	Partially addresses the gap through simulation and planning, but lacks runtime adaptability

Before applying the quantitative WSM, the three concepts are compared at a structural level to highlight their qualitative differences and to ensure that all requirement categories are considered. Table 4.1 presents this comparison across key design features, including automation capability, validation, adaptability, and platform compatibility.

The comparison shows that Concept B consistently addresses the core requirements of automatic program generation, including waypoint gener-

ation, regripping, opposite-side handling, and pre-execution validation. In addition, it aligns with the required software platform by integrating with the drag&bot environment through the AnyRobot Interface.

Concept A, although feasible and simple to implement, relies entirely on manual programming. While it can be executed within the drag&bot environment as a manually constructed workflow, it does not provide automated planning, validation, or configuration-driven adaptability. As a result, it does not address the primary objective of reducing programming effort.

Concept C demonstrates advantages in simulation-based validation and CAD-driven planning. However, its dependency on accurate digital models and lack of runtime adaptability limit its flexibility. Furthermore, it is not compatible with the required drag&bot and AnyRobot-based execution environment.

Overall, the structural comparison indicates that Concept B provides the most comprehensive alignment with the defined requirements, supporting the quantitative evaluation presented in Section 4.6.

4.6 Concept Decision-Weighed Sum Model (WSM)

4.6.1 Methodology

The concept selection in this work is performed using the Weighted Sum Model (WSM), a structured multi-criteria decision-making method widely used in engineering design. The WSM enables quantitative comparison of alternative concepts by evaluating them against a defined set of criteria derived from the system requirements.

Each evaluation criterion is assigned a weighting factor based on its relative importance in addressing the research objectives. The concepts are then evaluated against each criterion using a consistent scoring scale from 1 to 5, where 1 indicates minimal or no satisfaction and 5 indicates a high level of satisfaction. The overall score for each concept is obtained by multiplying the assigned weight with the corresponding score and summing the results across all criteria.

This method ensures that the concept selection process remains transparent, systematic, and aligned with the functional, non-functional, and system-level requirements defined in Chapter 3. It also reduces subjectivity by providing a structured framework for comparing alternative solution approaches.

4.6.2 Evaluation Criteria and Weights

The evaluation criteria are derived directly from the functional requirements (FR-01 to FR-07), non-functional requirements (NFR-01 to NFR-05), and software platform requirements (SR-01 to SR-03) defined in Chapter 3. The assigned weights reflect the relative importance of each criterion in addressing the identified research gap.

Programming Effort Reduction (25%) This criterion carries the highest weight as it directly addresses the primary research objective of reducing manual robot programming effort. In flexible manufacturing environments, frequent changes in part geometry require repeated program modifications, making manual programming a major limitation. Therefore, approaches that minimise or eliminate manual waypoint definition are prioritised.

Automated Regrip and Opposite-Side Handling (20%) Regripping and opposite-side bending are among the most complex and time-consuming operations in robotic sheet metal bending. These operations are often handled manually in conventional systems and significantly impact process efficiency [8], [9]. This criterion evaluates the extent to which a concept supports automated handling of these operations.

Pre-Execution Validation and Safety (20%) This criterion evaluates the ability of the system to detect infeasible or unsafe conditions prior to execution. It includes validation of reachability, orientation feasibility, and handling constraints, as well as the ability to perform corrective actions such as replanning or safe termination. This is aligned with safety and robustness requirements defined in NFR-03, NFR-04, and NFR-05.

Adaptability to Changing Workpiece Geometry (15%) This criterion assesses the flexibility of the system in handling variations in workpiece geometry without requiring significant reprogramming. Configuration-driven approaches that allow parameter modification without code changes are considered more adaptable and better suited for dynamic production environments.

Platform Compatibility (10%) This criterion evaluates whether the concept can operate within the required software environment, specifically the

drag&bot runtime and the AnyRobot Interface, as defined in SR-01 and SR-02. Concepts that do not align with these platform requirements are considered less feasible for implementation within the scope of this work.

Operator Usability (10%) This criterion evaluates the level of user interaction required during operation. Systems that minimise operator involvement and reduce the need for specialised programming knowledge are considered more user-friendly and suitable for industrial deployment, in accordance with NFR-01.

4.7 Result of the weighted Sum Model

Table 4.2 presents the complete WSM evaluation with score justifications and weighted totals for all three concepts

4.8 Analysis of results and Concept Decision

The results of the Weighted Sum Model (WSM) indicate a clear ranking among the proposed concepts. Concept B achieves the highest score (4.00), followed by Concept C (2.95), while Concept A obtains the lowest score (1.25). This reflects their relative effectiveness in satisfying the functional, non-functional, and platform requirements defined in Chapter 3.

Concept B performs strongly across all criteria. It reduces manual programming effort by generating robot programs from configuration parameters and supports automated regripping and opposite-side handling through dedicated modules. Pre-execution validation enables detection of infeasible conditions with support for replanning or safe termination. In addition, its configuration-driven structure allows adaptation to different workpiece geometries without code modification. The concept is fully compatible with the required platform, operating within the drag&bot environment via the AnyRobot Interface.

Concept C demonstrates moderate performance through CAD-based offline programming. It enables simulation-based validation and reduces manual teaching effort; however, it depends on accurate digital models and requires reconfiguration for each geometry variation. Furthermore, it lacks runtime adaptability and is not compatible with the required drag&bot and AnyRobot environment.

Concept A shows the lowest performance. Although it can be implemented within drag&bot as a manually constructed workflow, all motions

Table 4.2: Weighted Sum Model (WSM) evaluation of Concept A, Concept B, and Concept C

Evaluation Criterion	Wt.	Concept A — Justification	Sc.	Concept B — Justification	Sc.	Concept C — Justification	Sc.
Programming Effort Reduction	25%	All waypoints and process logic defined manually; no reduction in programming effort	1	Waypoints and process logic generated automatically from configuration parameters; minimal manual intervention required	4	CAD-based programming reduces manual teaching but requires model setup and adjustment for each variant	3
Automated Regrip and Opposite-Side Handling	20%	No automated regrip or opposite-side handling; fully operator-dependent	1	Automated regrip and opposite-side handling implemented through rule-based modules	4	Regrip and handling planned in simulation but not dynamically adapted during execution	3
Pre-Execution Validation and Safety	20%	No integrated validation; infeasible conditions detected only during execution	1	Pre-execution validation detects infeasible conditions and supports replanning or safe termination	4	Validation performed in simulation environment; no runtime validation during execution	3
Adaptability to Changing Workpiece Geometry	15%	Manual reprogramming required for each geometry change; low adaptability	1	Configuration-driven approach allows adaptation through parameter changes without code modification	4	Requires modification of CAD models and re-simulation for each new geometry	3
Platform Compatibility	10%	Can be executed within drag&bot as a manual workflow but does not utilise the AnyRobot Interface; partially satisfies platform requirements	2	Fully compatible with drag&bot runtime and AnyRobot Interface as defined in SR-01 and SR-02	4	Not compatible with drag&bot runtime or AnyRobot Interface; does not satisfy platform requirements	1
Operator Usability	10%	Requires expert knowledge and continuous manual input; high operator dependency	2	Minimal operator interaction; system handles planning and execution logic	4	Moderate usability; reduced robot expertise required but requires CAD knowledge	3
Weighted Total Score	**100%**	Concept A	**1.25**	Concept B — SELECTED ✓	**4.00**	Concept C	**2.95**

and process steps must be defined by the operator. This results in high programming effort and limited scalability. The absence of automatic planning, validation, and decision-making prevents it from addressing the defined research objectives.

Based on this evaluation, Concept B is selected as the most suitable approach. It provides the best balance between automation, adaptability, and system integration within the defined scope. The detailed implementation of Concept B is presented in Chapter 5.

5 Implementation

5.1 Introduction and scope

The product development phase translates the selected concept into a fully realised technical system. Based on the structured concept evaluation presented in Chapter 4.6, Concept B was selected for implementation, as it provides a feasible and systematic solution to the research problem while satisfying the defined functional, non-functional, and software platform requirements outlined in Chapter 3. In particular, Concept B enables automatic robot program generation, integrated constraint validation, and seamless operation within the drag&bot runtime environment.

The objective of this chapter is to present the complete implementation of Concept B as an executable software system for automated press brake sheet metal bending. The system generates robot programs automatically from geometric input parameters and configuration data, eliminating manual waypoint definition and reducing operator intervention. The implementation focuses on ensuring deterministic behaviour, repeatability, and safe execution through structured validation and modular system design. The implementation is structured around four key domains. First, the system architecture defines the layered module structure and underlying design rationale. Second, the interface specification describes data exchange between internal modules and communication with the drag&bot runtime via the AnyRobot Interface. Third, a Failure Mode and Effects Analysis (FMEA) evaluates potential system risks and corresponding mitigation strategies. Finally, the software implementation record documents the core algorithms, configuration structures, and their traceability to the requirements defined in Chapter 3.

The system operates entirely within the drag&bot virtual simulation environment. Communication with the robot runtime is established through the AnyRobot Interface using an authenticated WebSocket connection, ensuring compliance with SR-01 and SR-02.

All process parameters are loaded from a central YAML configuration file at the beginning of each execution cycle. This enables adaptation to different workpiece geometries without modifying the source code, satisfying NFR-02.

Operator input is provided through an interactive 3D sheet bending viewer integrated into the drag&bot operator cockpit. This interface improves us-

ability and reduces the likelihood of incorrect input selection.

Ubuntu Linux is used as the development platform due to its compatibility with ROS-based systems and stable integration with the drag&bot environment [24].

Figure 5.1 illustrates the overall system architecture and the sequence of operations from session initialization to bending execution.

The remainder of this chapter is organised as follows. Section 5.2 presents the system architecture, including the layered design, module organisation, and data flow. This is followed by the communication stack, configuration architecture, and operator interface. The simulation environment and spatial zone classification are then described. Subsequently, each functional subsystem is presented in execution order, including pre-execution safety validation, workpiece orientation strategy, grip planning, regrip handling, opposite-side bending, and trajectory generation. The chapter concludes with a Failure Mode and Effects Analysis and a requirement-to-implementation mapping.

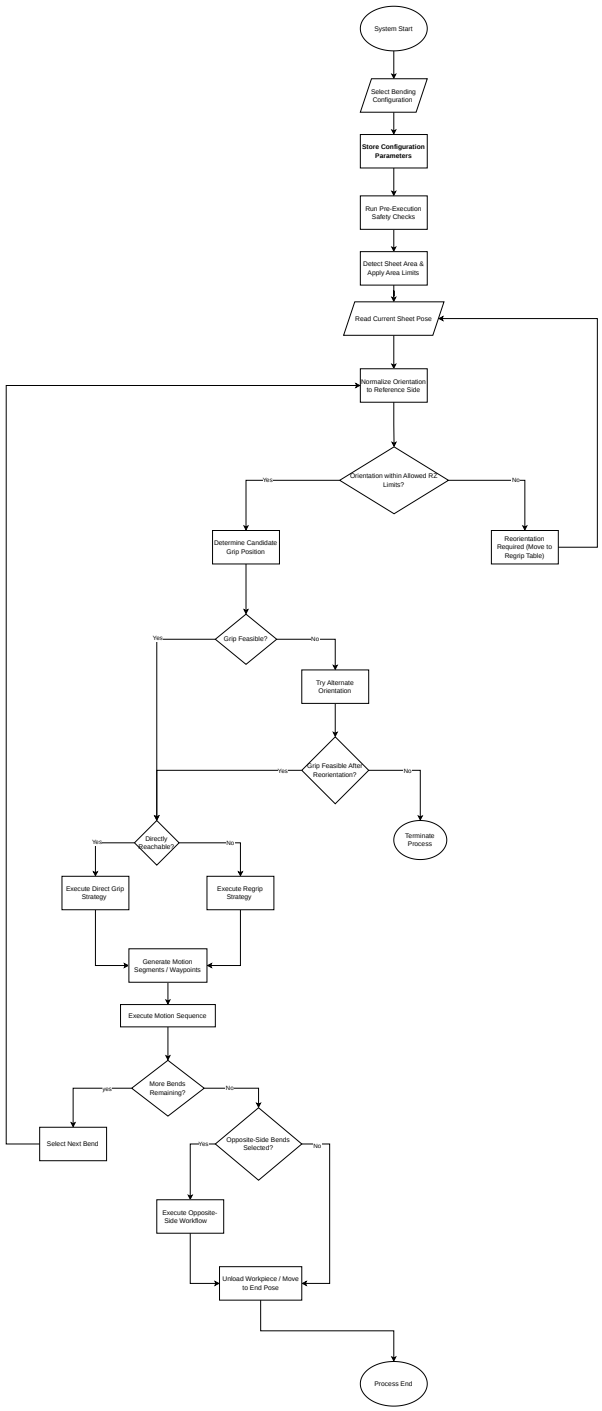


Figure 5.1: System architecture

5.2 System architecture

5.2.1 Layered Architecture Design and data flow

The system adopts a layered architecture in which each module depends exclusively on the module in the layer directly beneath it. This architectural pattern is selected because the press brake automation context imposes

three key constraints that cannot be satisfied by an unstructured design: independent testability of constraint evaluation, maintainability and safe replaceability of components, and strict enforcement of safety ordering. Testability is achieved by enabling modules to be evaluated using synthetic pose data and YAML-based configuration inputs, without requiring an active WebSocket connection. This isolation is essential for verifying orientation classification and grip feasibility computations, which cannot be reliably validated from live system behaviour alone.

Replaceability is ensured through modular separation, where modifications to the grip planning algorithm are confined to the constraint evaluation layer. The planning and motion layers remain independent of these modules, supporting maintainability and satisfying NFR-02.

Safety ordering is enforced structurally by implementing pre-execution validation in a dedicated layer that is executed prior to any motion generation. This guarantees that safety checks cannot be bypassed or reordered, satisfying FR-07 and NFR-03.

Five distinct layers are defined within the system architecture, each with a clearly assigned responsibility. Layer 1 (Input and Configuration) is responsible for acquiring all required parameters from the input file, along with the operator's bend-side selection, prior to the establishment of any WebSocket connection. Layer 2 (Geometry and Constraint Evaluation) performs the computation of geometric and physical constraints based on input pose data, while remaining strictly limited to evaluation tasks and producing no motion commands. Layer 3 (Planning and Decision Logic) determines the overall execution strategy and coordinates the interaction between Layers 2 and 4, without directly generating motion instructions. Layer 4 (Motion Generation and Communication) serves as the exclusive interface for robot communication, generating LIN and PTP waypoints, converting them into JSON format, and transmitting them via an authenticated WebSocket connection. Layer 5 (Safety and Validation) is implemented as an independent top-level layer that is executed prior to any motion generation and enforces mandatory validation and operator confirmation, preventing further execution until all safety conditions are satisfied. The complete architectural structure is illustrated in Figure 5.2.

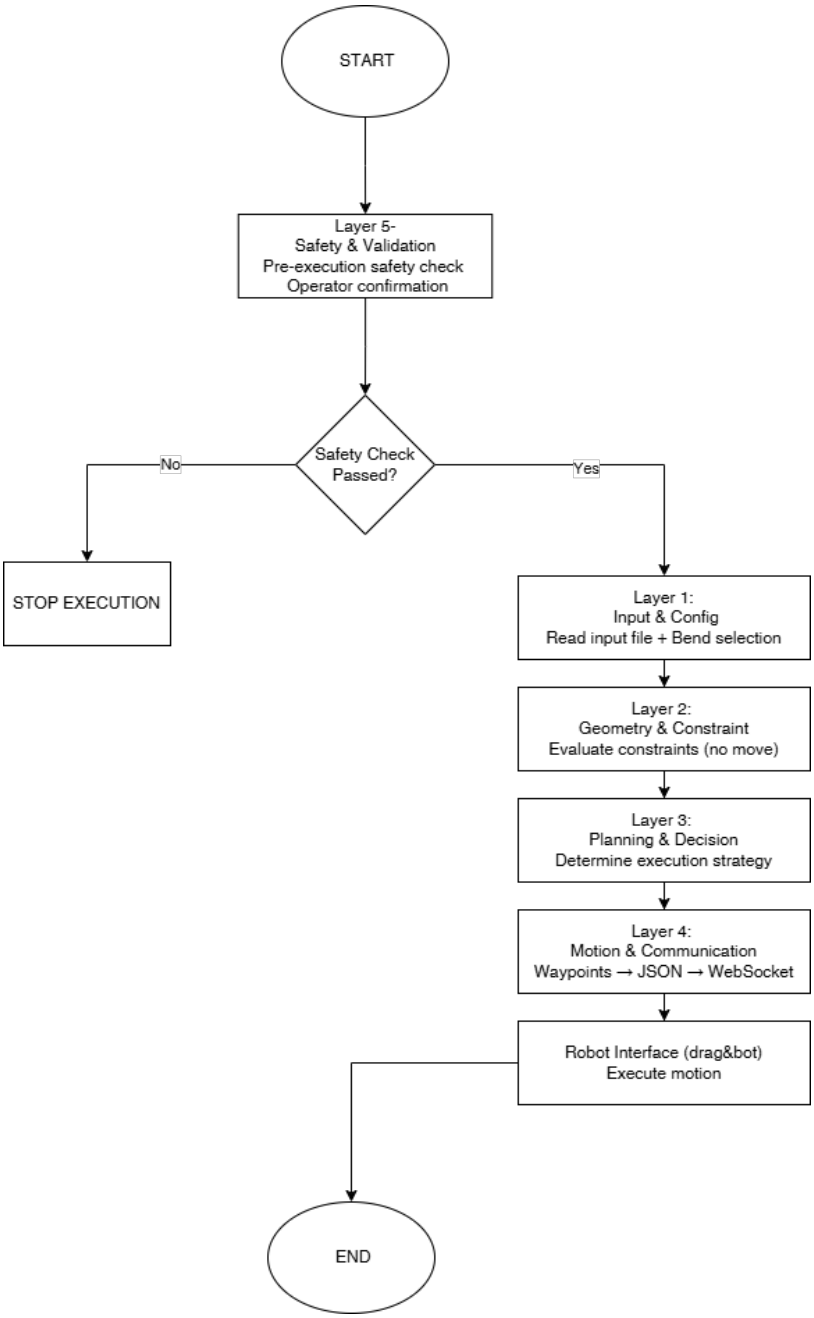


Figure 5.2: Five layer architecture

The progression of data throughout a bending cycle follows a structured transformation across the system layers. Initially, sheet geometry, bend-side selection, and tool parameters are provided as input. These inputs are processed within the constraint evaluation layer, where raw pose data is transformed into a validated pickup orientation and feasible grip configuration. Based on this information, an execution strategy is determined and translated into motion commands, which are encoded in JSON format and transmitted to the runtime environment.

The reliability of the overall process depends on several critical transfor-

mation stages. Any error introduced at these stages propagates through subsequent computations and affects execution outcomes. In particular, accurate orientation determination is essential for successful workpiece acquisition, while grip feasibility evaluation ensures stable handling during transport. During trajectory generation, a balance must be maintained between path accuracy and controller limitations, requiring controlled reduction of trajectory points while preserving motion characteristics.

5.2.2 Module Overview and dependency Discipline

The implementation consists of software modules organized across the five system layers. Table 5.1 summarizes each module in terms of its layer assignment, functional role, and primary responsibility. Each module is defined with a single primary responsibility to ensure clear fault attribution during failure analysis. Assigning multiple responsibilities to a single module would make it difficult to associate a specific failure mode with a well-defined functional unit.

Table 5.1: Software modules across system layers and their primary responsibilities

No.	Layer	Module	Primary Responsibility
1	L1	Session launcher	Coordinates YAML loading, 3D viewer initialisation, and session start-up before runtime connection.
2	L1	Configuration reader	Loads and validates configuration data and provides a consistent parameter source for the remaining modules.
3	L5	Safety checker	Enforces operator confirmation and safety validation before execution begins.
4	L2	Zone classifier	Classifies tracker data into spatial zones and determines admissible orientation strategies.
5	L2	Orientation selector	Determines the pickup orientation on the basis of geometric evaluation of the workpiece.
6	L2	Face orientation converter	Converts tracker pose data into a reference-face representation using rotation transformations.

No.	Layer	Module	Primary Responsibility
7	L2	Grip position planner	Computes feasible grip positions while considering edge clearance and holding constraints.
8	L2	Pickup orientation validator	Verifies tool alignment and applies corrective transformations where required.
9	L4	Trajectory generator	Generates waypoint sequences with path refinement, sampling control, and singularity avoidance.
10	L4	Regrip motion executor	Executes regrip operations, including transport, release, and re-acquisition.
11	L3	Primary bending orchestrator	Coordinates execution flow across orientation, planning, and motion modules.
12	L3	Regrip sequencer	Determines transition sequences between grip configurations.
13	L3	Opposite-side runner	Evaluates and executes alternative orientation strategies when required.
14	L3	Multi-stage regrip loop	Handles iterative repositioning for complex bending sequences.
15	L3	Post-bend unload sequencer	Controls unloading of completed workpieces to the designated buffer location.
16	L4	Waypoint data structure	Encapsulates robot motion parameters, including pose, velocity, and blending information.
17	L4	Digital IO handler	Manages vacuum control and safety-related input/output signals.
18	L4	WebSocket client	Handles authentication, session management, and message transmission to the runtime interface.
19	L4	Command type definitions	Defines motion categories, coordinate conventions, and JSON message formats.

Three structural properties ensure that the architectural constraints are consistently enforced across the system. First, the WebSocket client and the command type definitions together constitute the exclusive communication layer. No module outside Layer 4 constructs WebSocket messages or performs direct network operations. As a result, authentication handling and connection recovery are implemented centrally and remain consistent across all runtime interactions. Second, the

safety checker is positioned within Layer 5, which is not imported by any lower layer. This separation prevents planning or execution modules from invoking safety validation during runtime. Instead, safety checks are enforced strictly at the beginning of a session, ensuring that they cannot be bypassed or reordered during execution. Third, the trajectory generator serves as the sole interface for dispatching motion commands to the runtime. All waypoint sequences are routed through this module, ensuring that workspace constraints and singularity checks are consistently applied before execution.

5.2.3 Communication Stack, Authentication interface Specification

All runtime communication passes through the AnyRobot Interface, as it is the only supported external API for drag&bot. Direct ROS client libraries are excluded because they would couple the planning software to a specific ROS version, violating NFR-02 [20] [24]. The AnyRobot Interface translates JSON over a persistent WebSocket channel into native ROS operations using the RosBridge v2 protocol, enabling the entire planning software to run in standard Python 3 without a ROS installation on the planning machine.

WebSocket (RFC 6455) is chosen over HTTP because it supports a persistent connection that allows continuous data transfer. In contrast, HTTP requires a new TCP connection for each request, which introduces a delay of approximately 3–15 ms per batch. When sending a trajectory of 400 points in batches of 50 commands, these delays accumulate to 24–120 ms. This added delay can cause the robot controller's command buffer to run empty, leading to interruptions during motion execution. By maintaining a continuous connection, WebSocket avoids this overhead and ensures smooth and uninterrupted command transmission.

Figure 5.3 presents an overview of the system communication architecture. The planning software establishes a persistent WebSocket connection to the robot controller via a defined network endpoint (e.g., `ws://192.168.x.x:9091`). Communication is organized through a set of dedicated interfaces, each responsible for a specific function within the system. The Ros Bridge v2 interface is used to transmit motion commands, while the `rosauth` service ensures secure authentication using the SHA-512 hashing algorithm. This method encodes authentication data into a non-reversible format, preventing the transmission of sensitive information in plain text. Motion trajectories are transmitted in batches via the `/command_list` interface, and real-time object information is retrieved through `/dnb_object_manager/list_objects`. Gripper control is performed using `/robot/set_digital_ios`, whereas object attachment and detachment are handled through the corresponding manager interfaces. User interactions, such as confirmations and prompts, are managed via the `human_interaction/request_hi` interface. The YAML configuration is not part of the runtime communication. It is loaded once during system initialization and shared across the relevant modules. This

approach ensures that missing or incorrect parameters are detected early, before any communication with the robot is initiated [25].

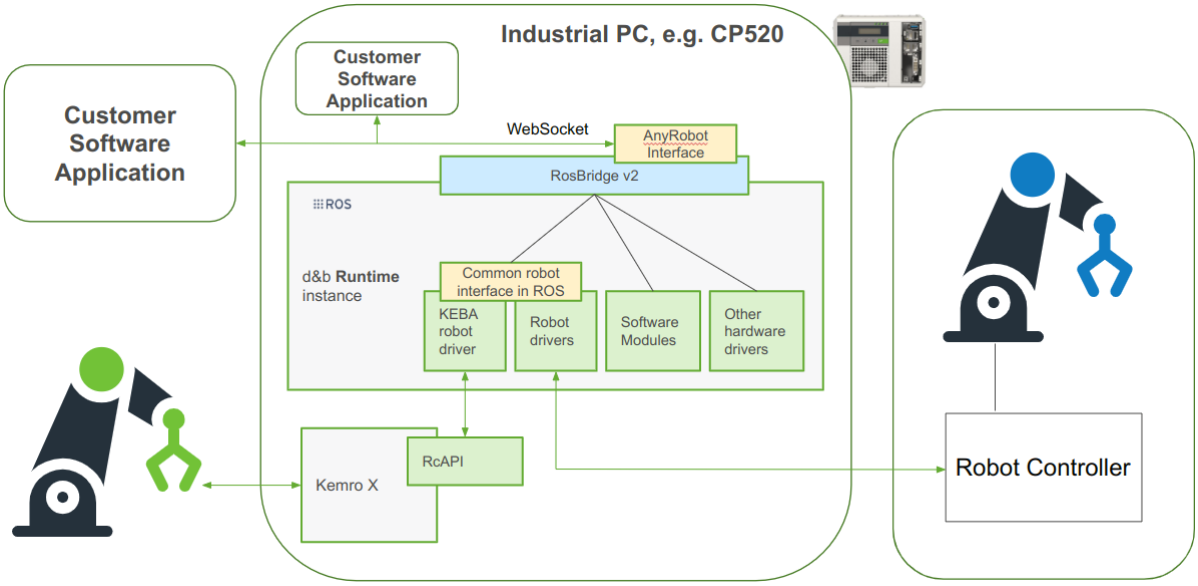


Figure 5.3: Communication architecture of the implemented system, illustrating the interaction between the planning software, the drag&bot runtime environment, and the robot controller (adapted from [20]).

On connection establishment, the WebSocket client performs the rosauth SHA-512 authentication handshake, because the drag and bot WebSocket port is accessible to any process on the local network and an unauthenticated endpoint would allow arbitrary motion command injection. The MAC is computed by hashing with SHA-512 concatenation of the shared runtime secret, the interface identifier, destination, a fixed nonce, session timestamps, and the access level string.

The authentication token is computed based on the procedure defined in the drag&bot AnyRobot Interface specification [20]:

$$\text{MAC} = \text{SHA-512}(\text{secret} \parallel \text{id_string} \parallel \text{dest} \parallel \text{nonce} \parallel \text{t} \parallel \text{level}) \tag{5.1}$$

SHA-512 is selected due to its strong cryptographic properties, as its 512-bit output makes pre-image and collision attacks computationally infeasible within the operational context of this application. The shared secret is stored as an environment variable to prevent exposure to version control systems. The concatenation order is strictly defined by the AnyRobot interface specification and must be followed precisely. Any deviation from this order results in a mismatch of the generated hash, causing the runtime system to reject the connection.

Listing 5.1 presents an illustrative Python implementation of the authentication token computation described in Equation 5.1, following the drag&bot AnyRobot Interface specification [20].

```
1 import hashlib
```

```python
import os

secret = os.getenv('RUNTIME_SECRET')

id_string = 'library'
dest = 'interface'
nonce = 'example_nonce'
timestamp = 0
role = 'user'

raw = secret + id_string + dest + nonce + str(timestamp) + role

mac = hashlib.sha512(raw.encode()).hexdigest()

auth_msg = {
    'operation': 'auth',
    'mac': mac,
    'nonce': nonce,
    'timestamp': timestamp,
    'role': role
}
```

Listing 5.1: Illustrative Python implementation of SHA-512-based authentication token computation following the drag&bot AnyRobot Interface specification.

Figure 5.4 shows the robot control panel used for system interaction and monitoring.

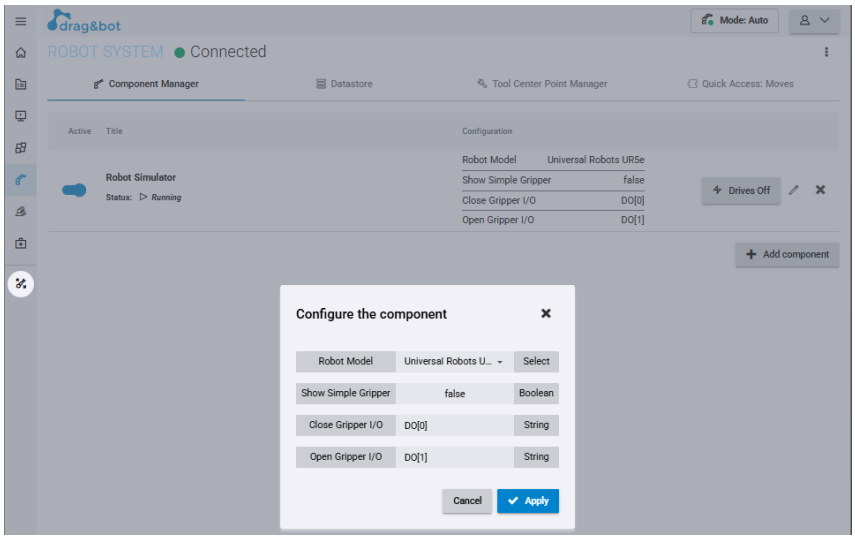


Figure 5.4: Robot control panel interface used for system operation and monitoring.

5.2.4 Configuration Architecture and YAML File Structure

The YAML configuration file serves as the central entry point for all process-related parameters and provides a flexible mechanism for adapting the system to different workpieces without requiring modifications to the underlying source code, thereby fulfilling NFR-02. At the beginning of each session, the file is parsed once, and the resulting configuration data is stored as a dictionary that is shared across all modules in Layers 2 to 4. The structure of the file is organized into six clearly defined top-level sections, each corresponding to a specific functional aspect of the system architecture, which supports clarity and maintainability. YAML is selected over JSON primarily due to its support for inline comments. This allows engineers to document the meaning and physical relevance of individual parameters directly within the configuration file, eliminating the need to consult external documentation. To ensure robustness and security, the configuration is loaded using a safe parsing method, which prevents the execution of unintended code in the event that the file is modified or corrupted. A key design decision concerns the handling of spatial consistency during the regrip process. Rather than defining approach and placement poses independently, all such poses are derived relative to a single reference pose defined for the orientation table. For example, the approach position is calculated by applying a fixed vertical offset to the placement pose. This design ensures that any change in the position of the orientation table requires only a single modification within the configuration file. In contrast, storing these poses independently could introduce inconsistencies: if the table position is updated but the corresponding approach position is not, the system may execute motions based on outdated spatial data. Such inconsistencies are particularly problematic because they may not be detected during configuration validation, as each parameter remains individually valid despite being collectively incorrect.

Listing 5.2 presents a generalised YAML configuration illustrating the key process, motion, and safety parameters without exposing system-specific or confidential information.

```
# =========================
# Robot configuration (generalised)
# =========================

robot:
  host: <robot_ip_address>
  port: <port_number>
  speed_mps: 0.5
  accel_percent: 20
  blend_percent: 15
  io_group: DO
  io_pin: 1
```

```yaml

# ========================
# Process and sheet definition
# ========================

process:
  sheet:
    length_m: 0.4
    width_m: 0.3
    thickness_m: 0.005
    mass_kg: 2.5

  grip:
    cup_count: 2
    min_edge_clear_m: 0.05
    min_spacing_m: 0.2
```

Listing 5.2: Generalised YAML configuration illustrating robot parameters, process definition, and motion and safety constraints.

5.2.5 Operator interface – interactive 3d sheet bending viewer

Operator input for each execution cycle is provided through an interactive three-dimensional sheet bending viewer developed in this work (hereafter referred to as the *3D viewer*), rather than a conventional text-based interface. This design decision is primarily motivated by the spatial nature of bend side selection. By presenting the workpiece directly as a 3D model, the operator can identify relevant surfaces and orientations more intuitively than by relying on abstract side labels. In practice, this reduces the likelihood of incorrect selections while keeping the interaction simple, as the operator performs only visually guided inputs and the system handles the underlying computations.

The 3D viewer is implemented as a web-based application using Plotly Dash and renders the workpiece as an interactive mesh derived from the corresponding STL file. It operates as a local service and is integrated into the drag&bot Operator Cockpit through the Custom URL panel mechanism. A new panel of type *Custom URL* is added to the dashboard via the dashboard configuration interface, where the local viewer address is specified as the target. Once configured, the viewer is rendered as an embedded panel within the cockpit, allowing the operator to interact with the 3D mesh and associated controls without leaving the drag&bot environment. The configuration is stored as part of the dashboard definition and remains available across sessions, ensuring a consistent setup for repeated operation.

This integration ensures that configuration, monitoring, and execution are performed within a single interface, eliminating unnecessary context switching and providing a more coherent workflow from the operator's perspective. The overall system setup used during development and validation, including the robot, press brake, and orientation table, is shown in Figure 5.5.

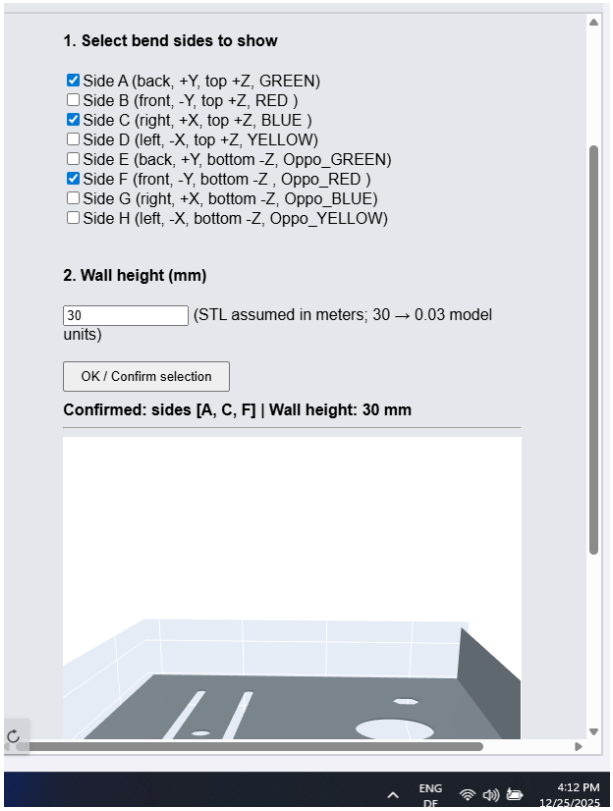


Figure 5.5: Operator workflow illustrating the sequential execution stages. Each stage must be completed before the next begins. All operator interactions occur within the drag&bot environment. *Source: own work.*

To accommodate different levels of operator expertise, the interface is divided into two modes. In basic mode, the operator selects from a predefined set of bending sides and specifies the required wall height. The system automatically prevents invalid combinations, particularly those that would require additional operations such as regripping, which are intentionally excluded at this level to simplify interaction. In contrast, expert mode exposes additional parameters, including geometric constraints, tool configuration, and pose definitions. However, changes made in this mode are not stored automatically and must be explicitly transferred to the configuration file, ensuring that persistent modifications are introduced in a controlled manner.

In parallel with user input, the system performs a geometric analysis of the imported mesh to identify internal features such as holes. These features are detected based on their orientation and their spatial relationship to the outer boundary of the sheet, and are subsequently classified according to their geometric characteristics.

The resulting feature information is used during grip planning to exclude unsuitable regions from consideration. The workpiece model used for this analysis is illustrated in Figure 5.6. Additionally, the workpiece mass is estimated using a standard material density, providing a conservative reference for evaluating whether a given gripping configuration is physically feasible.

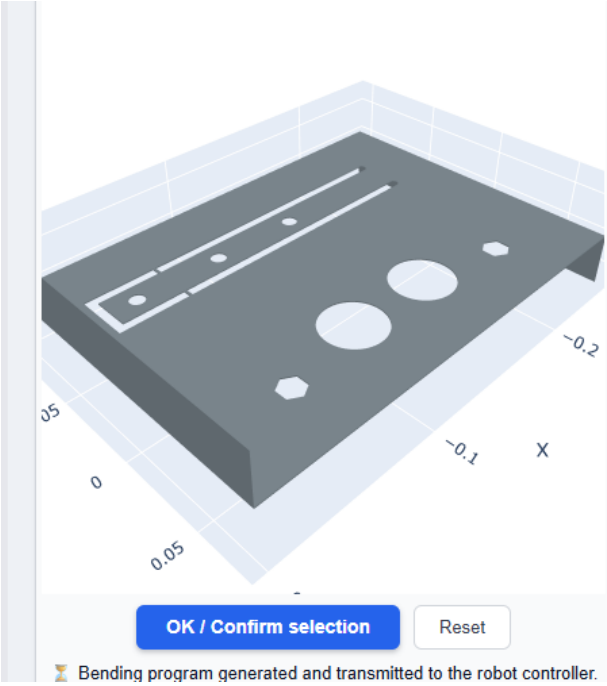
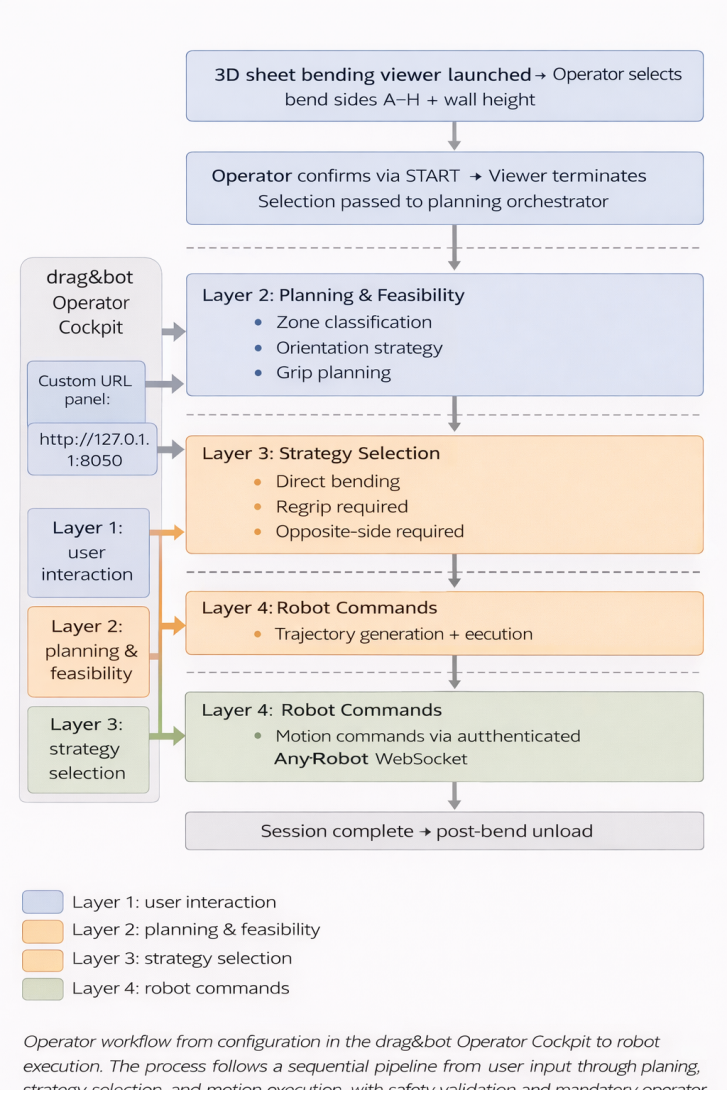


Figure 5.6: Sheet metal workpiece model used for geometry analysis. Internal cut-outs are detected and excluded from the grip candidate search region. Source: own work.

Once the operator confirms the selection, the defined parameters are transmitted to the planning module in a structured format. The viewer then terminates after a short delay. This behaviour is intentional, as it prevents further interaction while the robot is executing a trajectory, thereby avoiding inconsistencies that could arise if configuration changes were introduced during active motion.

The complete sequence of operator interaction and system processing is summarised in Figure 5.7, which illustrates the progression from initial configuration through planning, strategy selection, and final execution.



Operator workflow from configuration in the drag&bot Operator Cockpit to robot execution. The process follows a sequential pipeline from user input through planing,

Figure 5.7: Operator Workflow .Every gate must be passed before the next begins.All operator interactions occu within the drag and bot enviornment. *Source: own work.*

5.3 Development and Simulation Environment

With the architectural framework established in Section 5.2, a controlled environment is required to execute, test, and validate the developed system prior to deployment on physical hardware. This section presents the development and simulation environment used in this work, including the drag&bot platform, the robot model and tool configuration, and the spatial zone classification strategy used to structure the workspace.

5.3.1 Simulation Platform, Robot Model, and TCP Configuration

The drag&bot platform is used as the primary environment for system development, integration, and validation. It provides a virtual robot controller together with the

AnyRobot WebSocket Interface, enabling the planning software to interact with the robot runtime without requiring direct access to physical equipment. This supports safe testing, repeatability, and efficient iteration during development. The use of virtualised environments is consistent with current industrial trends in digital manufacturing and digital twin-based validation [20], [26].

All modules are implemented in Python and executed within an Ubuntu-based environment alongside the drag&bot runtime. Communication is established through a single authenticated WebSocket connection (e.g., `ws://<local-ip>:9091`), which is used for motion execution, IO control, object management, and operator interaction. This unified communication structure ensures consistent behaviour across all subsystems.

All session-specific parameters are initialised at the beginning of execution and explicitly passed between modules. This avoids hidden dependencies and ensures that all components operate on consistent data, supporting modularity and maintainability.

Within this environment, the Universal Robots UR5e is used as the manipulator model. The robot provides six degrees of freedom, a reach of approximately 850 mm, and a payload capacity of 5 kg, which are sufficient for handling the sheet metal workpieces considered in this work. The availability of a validated kinematic model within the drag&bot platform allows all generated motion targets to be evaluated for reachability and feasibility during simulation [20], [27].

The Tool Centre Point (TCP) used for all motion targets is defined in the drag&bot tool management system as `project_3_tcp`, in accordance with requirement SR-03. The TCP is specified using the ZYX Euler convention with offsets relative to the robot flange: $x = 0.000$ m, $y = 0.034$ m, $z = 0.194$ m, $RZ = -90.0°$, $RY = -75.0°$, and $RX = 0.0°$. These parameters ensure that all generated poses remain consistent with the physical gripper configuration. The TCP identifier is provided through the operator input, allowing future adjustments without modification of the underlying source code.

5.3.2 Virtual Cell Layout and Spatial Zone Classification

The simulated robotic cell within the drag&bot environment includes the robot, press brake, orientation table, and a tracked sheet object, which serves as the reference for all geometric computations. The spatial configuration of these elements reflects the intended physical setup, ensuring that simulation results are representative of real-world operation.

At the beginning of each execution cycle, the system retrieves the six-degree-of-freedom pose of the sheet from the object tracking service. The pose is expressed in the robot world coordinate frame, where the origin is located at the robot base. Accurate pose acquisition is a fundamental requirement in robotic manipulation systems and is widely addressed in modern robotics literature [3]. Based on this

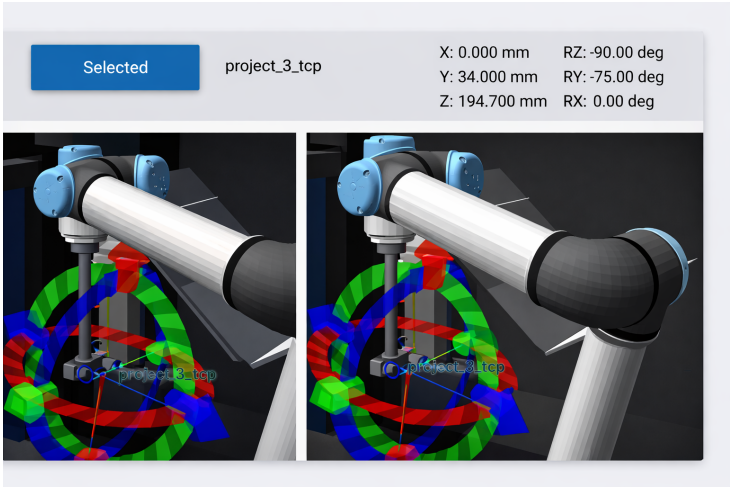


Figure 5.8: Tool Centre Point (TCP) configuration of the UR5e robot used for motion planning. *Source: own work.*

information, the position of the sheet centroid in the XY plane is used to classify the workpiece into one of four spatial zones.

The workspace is partitioned into four regions relative to the robot base. A centroid with negative X and positive Y corresponds to Zone 1 (rear-left), while negative X and negative Y defines Zone 2 (front-left). Positive X and positive Y correspond to Zone 3 (rear-right), and positive X with negative Y defines Zone 4 (front-right). The classification is based on threshold values of ± 10 mm in the X direction and ± 50 mm in the Y direction. These thresholds account for measurement uncertainty and reflect the physical layout of the robotic workspace [3].

Zones 3 and 4 lie within the primary reachable workspace of the robot and therefore allow direct execution of pickup and bending operations. In contrast, Zones 1 and 2 correspond to regions with limited reachability. In these cases, the system automatically initiates a repositioning step, moving the workpiece to an intermediate configuration within the reachable workspace before continuing execution. This ensures that all subsequent operations remain within kinematic limits without requiring operator intervention.

If the sheet centroid lies within the dead-band region defined by $|x| < 10$ mm or $|y| < 50$ mm, the classification is considered unresolved. This occurs near the coordinate axes, where small variations in tracking data may lead to inconsistent zone assignment. To prevent incorrect execution, the system terminates the current session and prompts the operator to reposition the workpiece before restarting.

Having defined the simulation environment, robot model, TCP configuration, and workspace segmentation strategy, the following sections describe the functional subsystems in the order of execution within a bending cycle, beginning with safety validation, followed by orientation determination, grip planning, and trajectory generation.

Algorithm 1 summarises the classification logic in an abstract form. The method

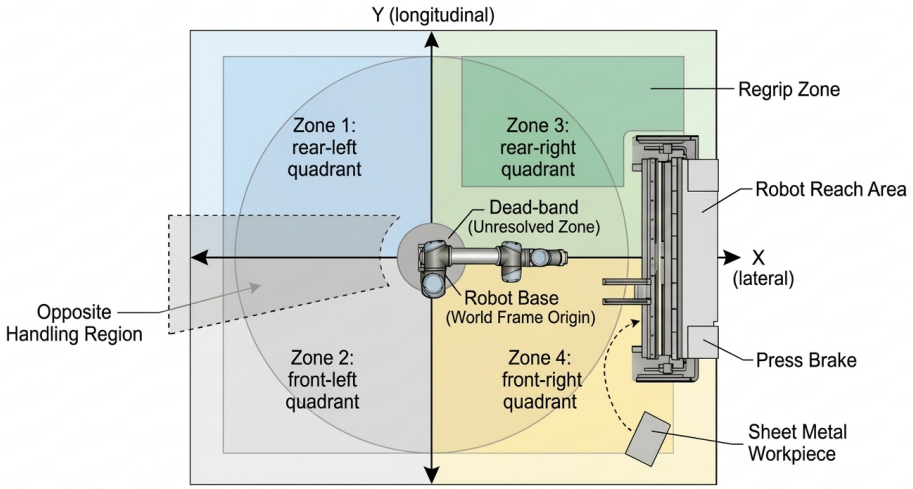


Figure 5.9: Spatial zone classification of the robot workspace based on the sheet centroid position. *Source: own work.*

Algorithm 1 Spatial zone classification based on sheet centroid position

Retrieve sheet centroid coordinates (x, y) in the robot world frame
if $|x| < x_{\text{th}}$ **or** $|y| < y_{\text{th}}$ **then**
    Return *unresolved zone*
else if $x < 0$ **and** $y > 0$ **then**
    Assign Zone 1
else if $x < 0$ **and** $y < 0$ **then**
    Assign Zone 2
else if $x > 0$ **and** $y > 0$ **then**
    Assign Zone 3
else
    Assign Zone 4
end if

relies on the sign of the centroid coordinates together with a threshold-based dead-band region to ensure stable classification under measurement uncertainty.

5.4 Pre-Execution Safety Validation

Safety validation is performed prior to motion generation to ensure that unsafe conditions and infeasible configurations are identified before robot execution, in accordance with NFR-03, NFR-04, and NFR-05 [3]. The system incorporates three complementary mechanisms: a fixture pose deviation check, a blocking operator confirmation dialog, and a runtime motion error monitoring module. In addition, all trajectory waypoints are generated and verified using robot kinematics to ensure reachability and feasibility before execution, thereby preventing invalid motion commands from being issued [3].

5.4.1 Fixture Pose Deviation Validation

The fixture pose deviation check retrieves the current poses of critical fixtures from the drag&bot object manager and compares them with reference poses obtained during initial cell calibration. The monitored elements include the primary sheet object at the press brake, the finished-part placement positions on the regrip table, and the placement and attachment references at the flip station. These fixtures are treated as fixed references within the planning system, as all grip positions, trajectory waypoints, and placement operations are computed under the assumption that their positions remain unchanged. Any deviation from these reference poses can therefore result in motion targets that are geometrically valid but physically incorrect.

The deviation check is applied uniformly across all spatial zones. Each fixture is evaluated using both positional and orientation criteria. The positional deviation is computed as:

$$\Delta_{pos} = \sqrt{(\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2} \tag{5.2}$$

and the orientation deviation is defined as:

$$\Delta_{ang} = \max\left(|\Delta RZ|, |\Delta RY|, |\Delta RX|\right). \tag{5.3}$$

A fixture is considered out of tolerance when Δ_{pos} exceeds 5 mm or Δ_{ang} exceeds 0.10 rad (5.7°). The positional threshold is selected to remain below the minimum clearance used in grip planning. This ensures that fixture misalignment does not produce misleadingly valid grip configurations. The angular threshold corresponds to the minimum deviation that would affect the accuracy of vacuum cup contact on the sheet surface.

The primary sheet object is evaluated using the same criteria as other fixtures. Since the planning system relies on the current sheet pose for all geometric computations, any deviation may lead to incorrect motion generation and is therefore treated as a configuration error.

If one or more fixtures exceed the defined thresholds, the system generates a structured warning listing all affected fixtures together with their measured positional and angular deviations. This warning is presented through a blocking human interaction (HI) dialog titled *Fixture position warning*, allowing the operator to either proceed with explicit acknowledgement or cancel execution for further inspection. The operator's decision is recorded, ensuring that responsibility for accepting out-of-tolerance conditions remains with the human operator.

If a fixture is not detected in the object manager response, it is treated as missing and automatically flagged as a validation failure.

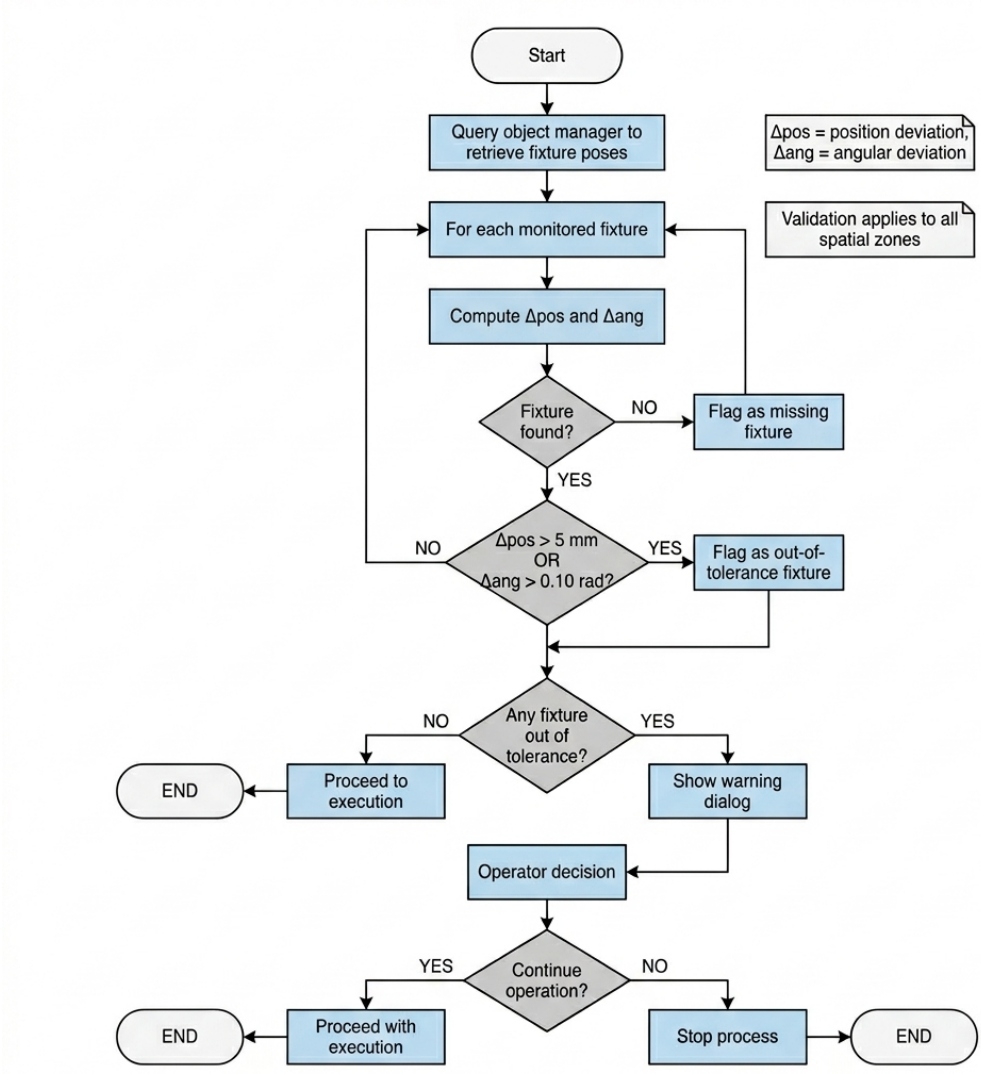


Figure 5.10: Fixture pose deviation validation workflow. Each monitored fixture is evaluated by comparing positional and angular deviations against defined thresholds. Out-of-tolerance or missing fixtures trigger a blocking warning dialog requiring operator confirmation. Validation is applied uniformly across all spatial zones. Source: own work.

5.4.2 Operator Authorization and Runtime Error Monitoring

The operator confirmation dialog is positioned as the first step in the planning orchestrator startup sequence, preceding all subsequent processes including fixture validation, visualization, and motion execution. The dialog enforces a mandatory decision: either initiate the automated workflow or cancel execution. The orchestrator does not proceed until a response is received from the HI service, ensuring that execution cannot begin without explicit operator authorisation. If the operator selects cancel, the process terminates without issuing any command to the robot runtime.

In addition to pre-execution validation, runtime supervision is implemented through a motion error monitoring module operating as a background subprocess.

This component subscribes to the `/command_result` ROS topic via an independent WebSocket connection, enabling continuous monitoring of execution outcomes without interrupting the main command flow. When a non-zero result code is detected, indicating a motion execution failure, the system immediately triggers a modal HI notification to inform the operator.

The monitoring module does not directly interrupt the execution pipeline. Instead, it provides immediate feedback through the HI interface and requires operator acknowledgement before further interaction is permitted. This ensures that execution anomalies are not overlooked while maintaining operator authority over subsequent decisions.

Together, these mechanisms establish a layered safety structure. Pre-execution validation ensures safe initial conditions, while runtime monitoring provides immediate feedback on execution failures. Both mechanisms utilise the drag&bot HI service, ensuring a consistent and controlled interface for safety-related interactions. With these safeguards in place, the subsequent section addresses the workpiece orientation strategy.

5.5 Workpiece Orientation Strategy

Following pre-execution safety validation, the first geometric step in each bending cycle is the determination of a valid pickup orientation for the sheet. This is required because the sheet may be placed in arbitrary orientations within the workspace, while only a subset of these orientations is kinematically feasible for the target bending operation. The orientation strategy evaluates the tracked sheet pose, applies a structured selection process, and produces a validated pickup pose for subsequent grip planning.

5.5.1 Zone-Based RZ Admissibility

The admissible range of sheet rotation about the Z-axis is defined by a zone-dependent window $[RZ_{\text{min}}, RZ_{\text{max}}]$. This constraint reflects the limited rotational capability of the robot end-effector within different regions of the workspace [3]. Since the reachable orientation range varies with the spatial position of the sheet, zone classification is performed prior to orientation evaluation. The admissibility limits are retrieved from the configuration to ensure that only kinematically feasible orientations are considered. This prevents the selection of geometrically valid but unreachable poses.

5.5.2 Pickup Orientation Validation

Before applying the selection strategy, the tracked pose is validated to ensure that the gripper approaches the sheet from above. This is evaluated using the condition

$\cos(RX) \cdot \cos(RY)$. If the value is below -0.10, the tool Z-axis is oriented downward and no modification is required. Otherwise, a corrective transformation is applied:

$$RZ' = \text{wrap}(RZ + 180^\circ), \quad RY' = -RY, \quad RX' = \text{wrap}(RX + 180^\circ) \tag{5.4}$$

This transformation ensures proper tool alignment with the sheet surface without modifying the pickup position.

5.5.3 Three-Level Orientation Selection Strategy

To ensure that a valid pickup orientation is always obtained, a three-level selection strategy is applied. The levels are evaluated sequentially, and execution terminates as soon as a valid orientation satisfying admissibility constraints is identified. This guarantees robustness while prioritising geometrically consistent and kinematically feasible configurations.

The workpiece is defined using four reference faces established during system commissioning, each associated with a fixed angular offset about the sheet's local vertical axis (0°, 180°, $+90^\circ$, -90°). These offsets allow the system to normalise arbitrary orientations into a consistent reference frame.

Level 1: Reference-Face Equivalent Orientation At the highest priority level, the tracked orientation is converted into a reference-face equivalent by removing the face-specific offset:

$$R_{\text{new}} = R_{\text{raw}} \cdot R_z(-d) \tag{5.5}$$

The resulting rotation matrix is decomposed into ZYX Euler angles [3], [28]:

$$R = R_z(\alpha) \cdot R_y(\beta) \cdot R_x(\gamma) \tag{5.6}$$

$$RZ = \text{atan2}(R_{10}, R_{00}), \quad RY = \arcsin(-R_{20}), \quad RX = \text{atan2}(R_{21}, R_{22}) \tag{5.7}$$

If the computed RZ lies within the admissibility window, this orientation is selected. This level is preferred as it ensures consistent reference alignment and reduces trajectory complexity.

Level 2: Detected-Pose Evaluation If Level 1 fails, the raw tracked orientation is evaluated directly. This accounts for cases where the initial placement is already feasible and avoids unnecessary transformations. If the raw RZ satisfies admissibility constraints, it is selected.

Level 3: Inverted-Pose Fallback If neither Level 1 nor Level 2 satisfies the admissibility condition, a fallback orientation is generated:

$$RZ' = \mathsf{wrap}(RZ \pm 180^\circ), \quad RY' = -RY, \quad RX' = -RX \tag{5.8}$$

This transformation expands the reachable orientation set and guarantees that a pickup configuration is always obtained. Since this orientation may not strictly satisfy admissibility constraints, it is marked as *admissibility not confirmed*. This status is propagated to downstream modules, allowing conservative execution strategies to be applied.

Algorithm 2 High-level orientation selection strategy

```
    Retrieve tracked sheet pose and active zone
 2: Load zone-specific admissibility window {R_Z^min, R_Z^max}
    Validate pickup orientation and apply correction if required
 4: Compute reference-face equivalent orientation
    if reference-face orientation is admissible then
 6:     Select reference-face orientation
    else if detected orientation is admissible then
 8:     Select detected orientation
    else
10:     Compute inverted orientation
        Select inverted orientation (admissibility not confirmed)
12: end if
    Forward orientation to downstream planning modules
```

Algorithm 2 summarises the implemented selection strategy at an abstract level without exposing implementation-specific details.

Strategy Outcome The proposed strategy guarantees that a valid pickup orientation is always determined while prioritising kinematic feasibility and geometric consistency. By integrating admissibility constraints with a structured fallback mechanism, the method directly supports pre-execution validation and reduces the likelihood of infeasible motion generation. The selected orientation is then passed to subsequent modules for grip planning and trajectory generation.

5.6 Workpiece Handling and Gripper Control

Once a valid pickup orientation has been determined, the system must establish a reliable physical interaction between the gripper and the sheet while maintaining consistency between the physical robot state and the virtual simulation. Any mismatch between the vacuum state of the gripper and the attachment state in the simulation leads to incorrect pose propagation and invalid trajectory computation

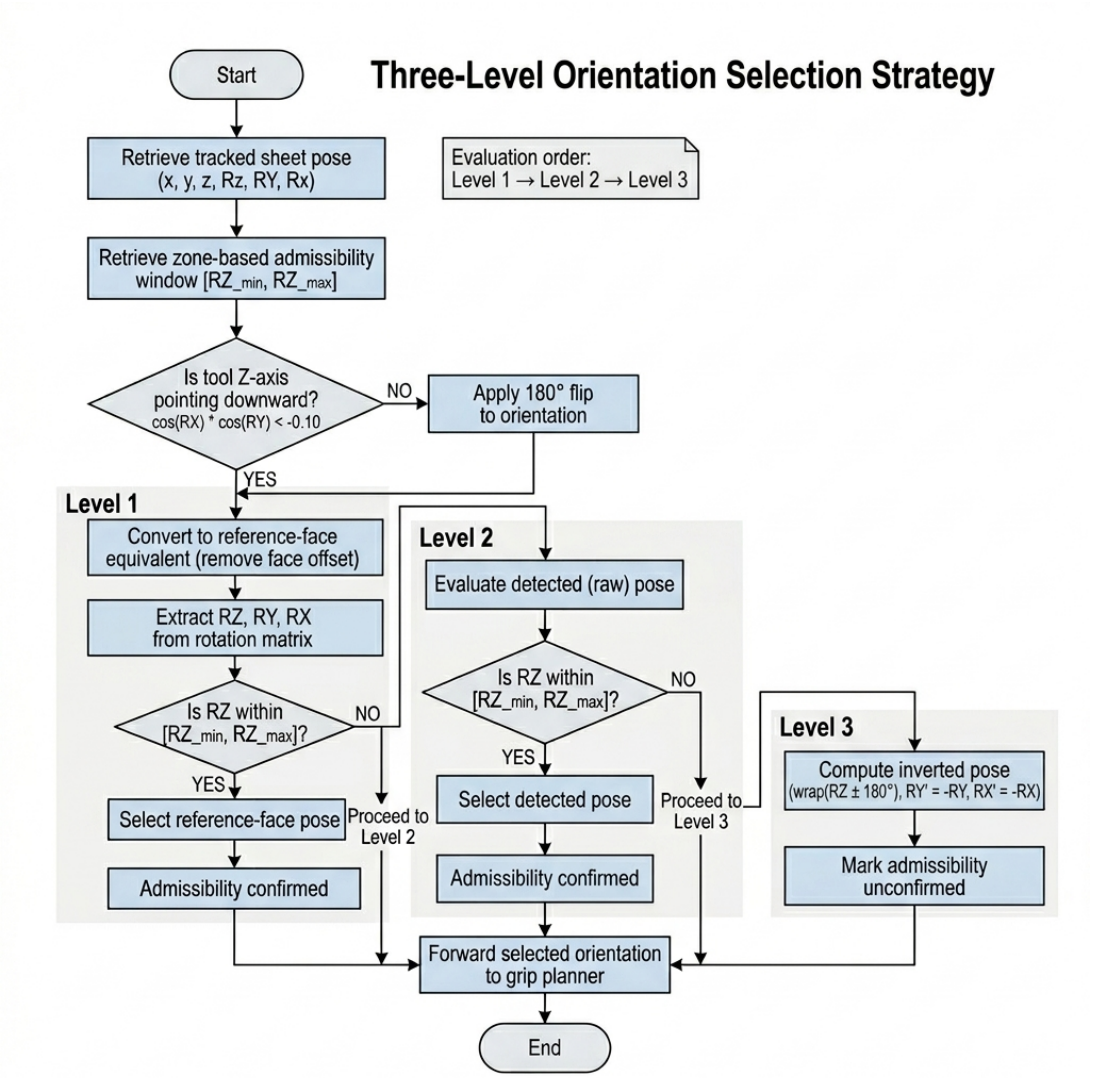


Figure 5.11: Three-level orientation selection strategy for determining a valid pickup pose under kinematic constraints. *Source: own work.*

in subsequent stages. This consistency is ensured through two coordinated mechanisms: digital IO control of the vacuum gripper and kinematic object attachment within the simulation environment.

5.6.1 Vacuum Gripper Digital IO Sequence

The vacuum gripper is actuated using two digital output signals mapped within the drag&bot IO management system. One signal activates the vacuum by closing the suction valve, while the other releases the vacuum by opening the exhaust valve. These signals are strictly mutually exclusive; simultaneous activation is avoided to prevent conflicting pressure states and potential hardware damage.

Control commands are issued via the `/robot/set_digital_ios` interface, which transmits structured IO instructions specifying the target output and its Boolean state. To ensure reliable gripping, a dwell period of approximately 250 ms is applied

after vacuum activation. This delay allows the suction cups to establish a stable seal with the sheet surface, which is necessary for maintaining grasp stability during subsequent motion [29].

A secondary delay of approximately 150 ms is introduced after each IO operation to ensure that the state transition propagates through the communication layer before further commands are issued. This prevents timing inconsistencies between command execution and runtime state updates.

5.6.2 Kinematic Object Attachment and Detachment

Within the simulation environment, the physical coupling between the gripper and the sheet is represented through kinematic attachment. When the gripper successfully establishes contact, the corresponding object is attached to the robot tool frame, ensuring that subsequent robot motions are reflected in the object pose. When the vacuum is released, the object is detached and remains fixed in its last known position.

Attachment is performed using the `/dnb_object_manager/attach` interface without enforcing a position override. This avoids introducing discontinuities between the tracked object pose and the simulated state, as the tracked pose may not exactly coincide with the instantaneous tool frame position. Position override is reserved only for controlled initialization procedures where explicit alignment is required.

Detachment is executed using the `/dnb_object_manager/detach` interface within the same reference frame. To maintain consistency, attachment and detachment operations are always executed in a fixed sequence together with the corresponding IO commands:

- Grasp sequence: vacuum ON $\rightarrow$ dwell $\rightarrow$ attach
- Release sequence: vacuum OFF $\rightarrow$ dwell $\rightarrow$ detach

This sequence is encapsulated within centralized helper routines that are invoked by all modules performing grasp or release operations. Centralizing this logic ensures that the physical and virtual system states remain synchronised at all times and prevents inconsistent conditions in which the vacuum state and attachment state do not correspond.

With physical grasping and state synchronisation established, the next step is to determine feasible suction cup positions on the workpiece surface.

5.7 Grip Position Planning and Feasibility Validation

Following the determination of a valid pickup orientation and the establishment of a consistent gripper control mechanism, the system must identify feasible suction cup placements on the sheet surface. Arbitrary placement is not sufficient, as it

may violate geometric constraints or fail to provide adequate holding force during handling. The grip planner therefore evaluates both geometric feasibility and physical stability to ensure a reliable and executable grasp configuration. This structured approach directly addresses the limitations identified in Chapter 2, where manual or heuristic-based gripping strategies lack consistency and adaptability. This approach enables deterministic and repeatable grip planning , which is essential for automated program generation in an industrial environment.

5.7.1 Sheet Geometry and Hole Detection

The sheet is modelled as a planar rectangular surface with uniform mass distribution, defined by its length, width, thickness, and mass obtained from the configuration. If the mass is not explicitly specified, it is estimated from the sheet volume and the material density defined in the configuration.

In addition to the outer boundary, internal features such as holes are extracted from the STL mesh during the geometry analysis stage. Each hole is represented by its centre position and radius in the sheet coordinate frame. These regions are explicitly excluded from the feasible gripping area, since suction cups cannot form a stable seal over discontinuities. This ensures that only physically valid contact regions are considered during grip planning.

5.7.2 Vacuum Holding Force Validation

For each candidate grip configuration, the system evaluates whether the available vacuum force is sufficient to retain the sheet under both gravitational and dynamic loading conditions. The contact area of a single suction cup is defined as

$$A_{\text{cup}} = \pi \left(\frac{d}{2}\right)^2 \tag{5.9}$$

The total available holding force generated by n suction cups is expressed as

$$F_{\text{vacuum}} = n \cdot \Delta P \cdot A_{\text{cup}} \cdot \eta \tag{5.10}$$

where ΔP represents the pressure differential and η accounts for efficiency losses due to leakage and surface conditions.

The required holding force is determined as

$$F_{\text{required}} = SF \cdot m \cdot \sqrt{a_{xy}^2 + (g + a_z)^2} \tag{5.11}$$

where SF is a safety factor and a_{xy}, a_z represent dynamic accelerations.

A candidate configuration is considered feasible only if

$$F_{\text{vacuum}} \geq F_{\text{required}} \tag{5.12}$$

This formulation follows standard principles of vacuum-based gripping and ensures that only configurations capable of maintaining stable contact under motion are considered for execution [30].

5.7.3 Grip Candidate Search and Selection

Candidate suction cup positions are generated using a multi-resolution grid-based search over the sheet surface. The process begins with a coarse discretisation to identify feasible regions, followed by progressive refinement to improve placement accuracy in promising areas. Additional candidates are generated near hole boundaries to maximise usable surface area while maintaining required clearances.

Each candidate position is evaluated against geometric constraints. The cup centre must satisfy minimum edge clearance requirements, avoid overlap with hole regions including a defined buffer zone, and comply with spacing constraints in multi-cup configurations. For the two-cup case, candidate positions are evaluated in pairs, with the inter-cup distance constrained to ensure sufficient grip span across the workpiece.

After geometric filtering, candidates are further evaluated using the vacuum holding force criterion described in Section 5.7.2. Only configurations satisfying both geometric and physical constraints are retained.

The remaining candidates are ranked using a scoring function that balances multiple criteria, including edge clearance, distance from internal features, and spatial distribution across the sheet. In multi-cup configurations, the scoring additionally promotes wider spacing between cups to improve stability during transport. This structured evaluation replaces heuristic placement strategies commonly found in manual programming approaches and ensures consistent selection of robust grip configurations.

The implemented search and selection procedure is summarised in Algorithm 3.

Algorithm 3 Grip position planning procedure

```
Load sheet geometry and gripping parameters
Validate vacuum holding feasibility
Generate candidate positions on the sheet surface
Filter candidates using geometric constraints
if two-cup configuration then
    Evaluate candidate pairs
end if
Score feasible candidates
Select best configuration
Compute A1, B1, R1 waypoints
Forward result to trajectory generation
```

The highest-scoring configuration is selected as the final grip position. Based on this selection, three key waypoints are derived: the approach pose (A1), the

attachment pose (B1), and the retreat pose (R1). These poses are computed relative to the sheet surface and tool geometry, ensuring a consistent and collision-free pickup sequence.

Figure 5.12 illustrates the complete grip planning pipeline, including force validation, candidate generation, constraint filtering, scoring, and waypoint derivation.

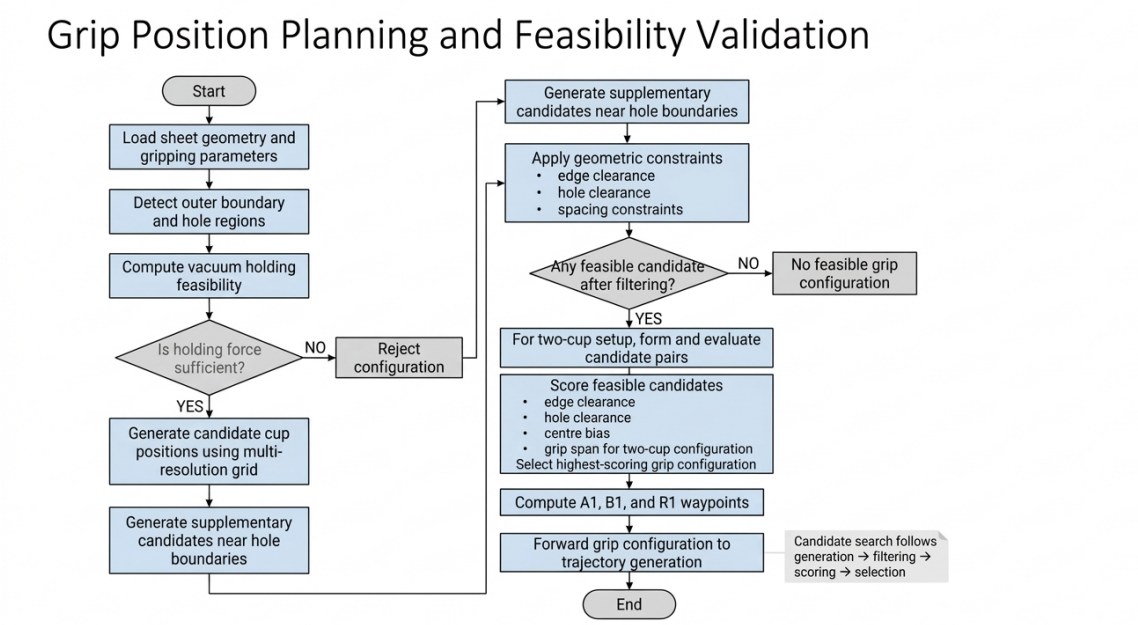


Figure 5.12: Grip position planning and feasibility validation workflow. The process integrates geometric filtering and vacuum force validation to ensure physically stable and executable grasp configurations prior to trajectory generation. Source: own work.

5.8 Regrip Planning and Execution

Even when a valid pickup orientation and a feasible grip configuration are established, the current grasp may not permit the robot to reach the required bending pose. This limitation arises when the target configuration lies outside the reachable workspace or when kinematic constraints, such as joint limits or proximity to singularities, prevent a valid motion solution [28]. To address this, the system implements an automatic regrip planning strategy that determines the required reorientation dynamically. This approach directly addresses the limitations identified in the state of the art, where regrip operations are typically predefined or manually programmed.

5.8.1 Regrip Decision Strategy

In contrast to conventional approaches based on fixed regrip sequences, the proposed system evaluates a discrete set of candidate workpiece orientations and selects a configuration that satisfies kinematic feasibility. This enables the system to determine both the required orientation and the number of regrip steps automatically.

The candidate set consists of discrete orientation increments corresponding to physically achievable rotations on the orientation table. For each candidate orientation, the system performs the following sequence:

- Update the workpiece pose according to the candidate orientation
- Determine a valid pickup orientation using the strategy described in Section 5.5
- Compute feasible grip positions using the planner described in Section 5.7
- Evaluate kinematic feasibility using inverse kinematics and trajectory validation

A candidate orientation is accepted only when all feasibility conditions are satisfied. The evaluation follows a sequential selection strategy in which candidates are tested in a predefined order, and the first feasible solution is selected. This ensures computational efficiency while maintaining robustness for real-time execution.

The number of required regrip steps is implicitly determined from the difference between the current orientation and the selected feasible configuration. This approach avoids unnecessary intermediate operations while ensuring that the final pose remains reachable and executable within the robot workspace.

Algorithm 4 Regrip orientation selection

```
Define candidate orientation set
for each candidate orientation do
    Compute updated workpiece pose
    Evaluate pickup orientation
    Compute grip configuration
    Check kinematic feasibility
    if feasible then
        Select orientation
        break
    end if
end for
Determine required number of regrip steps
```

Algorithm 4 summarises the decision process. The evaluation reuses the existing planning pipeline, ensuring consistency with orientation selection, grip planning, and trajectory generation.

5.8.2 Regrip Execution Sequence

Once a feasible orientation has been identified, the system executes a structured regrip sequence to realise the required reorientation. The sequence consists of the following steps:

1. Transport the workpiece from the bending station approach position to the orientation table deposit position while maintaining the current grasp
2. Release the workpiece through controlled vacuum deactivation and kinematic detachment
3. Move the robot to a retract standoff position to ensure collision-free clearance
4. Re-approach the workpiece at the updated orientation and perform the grasp sequence
5. Repeat the regrip cycle if additional orientation adjustments are required
6. Perform the final grasp and return the workpiece to the bending station

Each motion segment is executed using smooth trajectories to minimise dynamic loads on the gripper and ensure stable handling. Intermediate standoff positions provide safe transitions between grasping and releasing operations.

5.8.3 System Integration and Configuration

The regrip execution sequence operates within the existing motion framework and uses configuration-defined target positions, including deposit, re-grasp, retract, and approach poses. These parameters define the physical layout of the cell but do not determine the number of regrip steps.

The decision to trigger a regrip is handled by the planning orchestrator, which evaluates feasibility based on orientation admissibility and kinematic constraints. This separation between decision-making and execution ensures modularity and allows the regrip strategy to be extended without modifying the motion execution layer.

Compared to configuration-driven approaches, the proposed method improves adaptability by determining the minimum required number of regrip steps dynamically. At the same time, it maintains computational efficiency by evaluating a structured set of candidate orientations rather than performing global optimisation. This balance makes the approach suitable for industrial deployment, where predictable and robust behaviour is required.

5.9 Opposite-face Bending Workflow

After completing all primary-side bending operations, the system evaluates whether additional bends are required on the opposite face of the sheet. These operations cannot be executed directly from the initial pickup configuration, as the required face is not accessible to the press brake tool. Consequently, the workpiece must be reoriented through a controlled flip operation before continuing the bending process.

5.9.1 Pre-Flip Verification and Transition Correction

Before initiating the flip sequence, the system verifies that the current sheet pose corresponds to the expected reference configuration. This verification is performed using a wrap-safe six-degree-of-freedom pose comparison, with a positional tolerance of 2 mm and an angular tolerance of 1°.

This validation step is required because the sheet may have undergone regrip operations during the primary bending sequence. Any deviation from the expected pose would propagate errors into subsequent motion planning stages, leading to incorrect trajectory generation.

If the verification fails, a transition correction step is executed. The system first restores the reference orientation using the strategy described in Section 5.5. A controlled pickup and repositioning sequence is then performed to re-establish a consistent and verified alignment between the robot and the workpiece.

If the verification is successful, the correction step is omitted and the system proceeds directly to the flip sequence.

5.9.2 Sheet Flip Sequence and Opposite-face Bending Execution

The flip operation is performed using a dedicated station within the robotic cell. The workpiece is transferred through two predefined reference positions to achieve a physically feasible reorientation.

First, the robot transports the sheet to the flip station placement position while maintaining the current grasp. The workpiece is then released in a controlled manner. The robot subsequently moves to an intermediate standoff pose and re-approaches the sheet from the opposite side, establishing a new grasp that results in the flipped orientation.

This two-stage transfer is preferred over a direct in-gripper rotation. A direct rotation would require a continuous motion that exceeds the kinematic capabilities of the robot and may violate workspace or collision constraints [28].

Following the flip operation, the sheet is aligned such that the previously inaccessible face is oriented towards the press brake tool. The system then applies the same validation pipeline used for primary-side bending, including zone classification, orientation determination, grip position planning, and trajectory generation.

Reusing the same processing pipeline ensures consistent system behaviour across both primary and opposite-side operations. It also avoids the need for separate handling logic, thereby maintaining modularity and reducing system complexity.

Figure 5.13 illustrates the complete opposite-side bending workflow, including pre-flip verification, optional transition correction, flip execution, and subsequent bending operations.

After completing all opposite-side bends, any remaining staged operations are executed using the existing handling framework. The system then proceeds to the final unloading sequence.

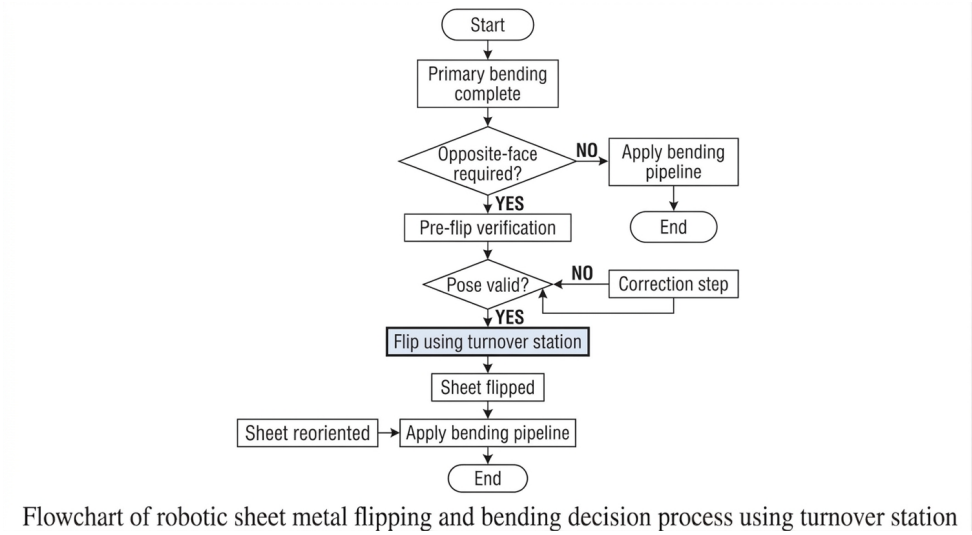


Figure 5.13: Opposite-face flip workflow enabling access to previously unreachable bending surfaces. Source: own work.

ints, and executed by the robot as a continuous motion from the start to the target pose.

5.10 Trajectory Generation

Trajectory generation constitutes the final stage of the planning pipeline, where validated task-level poses are transformed into executable robot motion. The input to this stage is an ordered sequence of tool centre point (TCP) poses obtained from upstream modules, including grip planning, regrip handling, and opposite-face operations. Each pose defines the desired end-effector position and orientation within the robot world frame.

Rather than executing direct point-to-point transitions, the system constructs a continuous and smooth trajectory composed of intermediate waypoints. This approach ensures kinematic feasibility, reduces dynamic loading, and improves execution stability during sheet handling.

5.10.1 Three-Phase LIN Trajectory Structure

Each motion segment is generated using a structured three-phase trajectory: lift, lateral transfer, and descent.

In the lift phase, the TCP is translated vertically to a predefined safe height to avoid collisions with the workpiece, fixtures, or surrounding environment. In the lateral transfer phase, the robot moves towards the target position while maintaining this clearance height. During this phase, the end-effector orientation is constrained to limit excessive joint rotations and reduce proximity to kinematic singularities. Finally, in the descent phase, the TCP is lowered to the target pose, where the required orientation for task execution is fully applied.

To further improve motion feasibility, an intermediate rotation pose is introduced above the target location. This ensures that major orientation adjustments are completed before the final approach, reducing the risk of instability near contact regions.

5.10.2 Waypoint Generation and Interpolation

The trajectory between successive poses is generated through interpolation in Cartesian space. For a start pose $\mathbf{T}_s$ and a target pose $\mathbf{T}_t$, the translational component is computed as

$$\mathbf{p}(\lambda) = (1 - \lambda)\mathbf{p}_s + \lambda\mathbf{p}_t, \tag{5.13}$$

where $\lambda \in [0, 1]$ is the interpolation parameter.

Rotational interpolation is performed using spherical linear interpolation (SLERP) in quaternion space, which provides smooth and constant-speed rotation between orientations [31].

To ensure smooth velocity and acceleration profiles, the interpolation parameter follows a minimum-jerk trajectory given by

$$s(u) = 10u^3 - 15u^4 + 6u^5, \tag{5.14}$$

which guarantees zero velocity and acceleration at the start and end of each motion segment [32]. Such profiles are widely adopted in robotic trajectory planning to achieve smooth and physically consistent motion behaviour [33].

By sampling this progression at discrete intervals, a sequence of intermediate TCP waypoints is generated that approximates the continuous trajectory.

```
1 def min_jerk(u):
2     return 10*u**3 - 15*u**4 + 6*u**5
3
4 waypoints = []
5 for k in range(N):
```

```
    u = k / (N - 1)
    s = min_jerk(u)
    p = (1 - s) * p_start + s * p_target
    r = slerp(r_start, r_target, s)
    waypoints.append((p, r))
```

Listing 5.3: Illustrative waypoint generation using minimum-jerk interpolation

The number of waypoints is determined based on the path length and a predefined resolution parameter, ensuring a balance between trajectory smoothness and computational efficiency.

```
distance = euclidean_dist(p_start, p_target)
N = max(2, math.ceil(distance / delta_s_max))
samples = [k / (N - 1) for k in range(N)]
```

Listing 5.4: Illustrative determination of waypoint count based on path length

5.10.3 Kinematic Mapping and Joint Execution

Each interpolated TCP waypoint is mapped to a corresponding robot joint configuration using inverse kinematics. For each waypoint, a feasible solution is selected based on continuity with the previous configuration, avoiding abrupt joint changes and ensuring smooth motion across the trajectory.

As the robot executes the sequence, all joints evolve continuously, resulting in a smooth end-effector motion that closely follows the planned Cartesian path.

5.10.4 Workspace Constraints and Singularity Handling

To ensure execution feasibility, each waypoint is validated against workspace constraints. The TCP pose is transformed into the corresponding flange pose, and its position is checked against the allowable workspace limits. If a waypoint exceeds these limits, it is projected onto the nearest feasible boundary.

Singularity avoidance is addressed by restricting orientation changes during the lateral motion phase. Large orientation adjustments are deferred to the approach phase, where the robot operates in a more stable configuration. This reduces the risk of excessive joint velocities and improves overall motion robustness.

5.10.5 Motion Execution and Path Reduction

The interpolation process generates a dense set of waypoints, which may exceed the processing capacity of the robot controller. To address this, a path reduction strategy is applied, retaining only those waypoints that exceed predefined positional or

rotational thresholds. This preserves the essential characteristics of the trajectory while reducing communication overhead.

The resulting waypoint sequence is transmitted as a series of linear motion commands, which are executed by the robot controller using blending techniques. This allows smooth transitions between consecutive waypoints without stopping, producing continuous and stable motion.

Overall, the trajectory generation module forms a deterministic transformation pipeline in which validated task-level poses are converted into executable robot motion through interpolation, kinematic mapping, constraint validation, and controlled execution.

5.11 System Integration, Execution Pipeline, and Conclusion

The developed system operates as an integrated pipeline that transforms operator-defined bending inputs into fully executed robot motion. This pipeline coordinates multiple modules, including task planning, trajectory generation, and execution control, ensuring consistent and deterministic system behaviour.

The execution begins with the operator defining the bending configuration, including the selection of bend sides and associated parameters. These inputs are processed by the planning module, which determines the sequence of operations required to complete the bending task. This stage establishes the order of primary-side bends, identifies necessary regrip operations, and determines whether opposite-face handling is required.

Following planning, the system enters the primary bending phase. Each bend is executed sequentially using validated motion strategies. For every operation, a feasible grip position is determined, orientation constraints are applied, and a corresponding trajectory is generated. Regrip sequences are performed when required to maintain stable and feasible grasp configurations throughout the process.

If bending operations on the opposite face are requested, the system transitions to the flip workflow. The sheet is transferred through the flip station, reoriented, and reattached so that the previously inaccessible face becomes available. The same planning and execution principles used in the primary phase are then applied, ensuring consistent handling across both sides of the workpiece.

All robot motion within these stages is executed through the trajectory generation pipeline described in Section 5.10. Each planned TCP pose is converted into intermediate waypoints, mapped to joint configurations through inverse kinematics, and executed as continuous motion using linear interpolation and blending. This ensures smooth and stable end-effector movement throughout the bending process.

After completion of all bending operations, the system performs the post-process unload as part of the execution pipeline. The robot attaches to the finished workpiece, transports it to the designated buffer station, and releases it in a controlled

manner. The robot then moves to a predefined safe start pose, restoring the system to a known configuration and ensuring safe operator access to the workspace.

The overall system therefore follows a structured execution pipeline:

- Operator input and bend configuration
- Planning of bending sequence and regrip strategy
- Execution of primary-side bending operations
- Regrip handling between operations
- Opposite-face handling via flip workflow (if required)
- Trajectory generation and motion execution
- Post-process unload and system reset

5.11.1 Conclusion

The presented system demonstrates a fully integrated approach to automated sheet metal bending using robotic manipulation. By combining structured task planning, constraint-aware trajectory generation, and controlled execution, the system achieves consistent and reliable handling of complex bending sequences.

The integration of regrip strategies and opposite-face handling enables the system to process multi-stage bending operations without manual intervention. The use of interpolation-based trajectory generation ensures smooth and stable motion, while workspace and orientation constraints maintain kinematic feasibility throughout execution.

Overall, the system provides a robust and deterministic framework for automated bending operations, supporting flexible task configurations while maintaining safety, repeatability, and execution stability. This integration forms the foundation for further evaluation through system reliability and failure analysis in the following section.

5.12 System Reliability and Failure Mode Analysis

To evaluate the robustness of the developed robotic bending system, a Failure Mode and Effects Analysis (FMEA) is conducted. This analysis identifies potential failure modes within the system, evaluates their impact on operation, and considers mitigation strategies implemented in the system design.

FMEA is a structured approach used to assess system reliability by analysing three key factors: severity (S), occurrence (O), and detectability (D). The combined effect of these factors is quantified using the Risk Priority Number (RPN), defined as

$$\text{RPN} = S \times O \times D \tag{5.15}$$

where each parameter is rated on a qualitative scale, typically from 1 (low) to 10 (high). Higher RPN values indicate higher priority risks that require mitigation.

5.12.1 Identified Failure Modes

The analysis focuses on key operational stages of the system, including gripping, trajectory execution, regrip handling, flip operations, and final placement. The identified failure modes are summarised in Table 5.2.

Table 5.2: Failure Mode and Effects Analysis (FMEA) of the robotic bending system

Failure Mode	Effect	S	O	D	RPN	Cause	Mitigation
Sheet unreachable during pickup	Robot stops mid-trajectory; session aborted	7	3	3	63	Invalid spatial placement or orientation outside admissible zone	Spatial zone classification and orientation constraints applied before motion; system halts with operator notification
Incorrect grip position (cup over hole)	Vacuum failure; sheet may fall during transport	8	2	2	32	Incomplete feature detection in hole map	Clearance offsets applied around detected holes; final validation ensures no suction cup overlaps with holes
Vacuum holding force insufficient	Sheet slips or drops during motion	8	2	1	16	Insufficient gripping force due to load or acceleration	Force feasibility check applied for each candidate grip; safety factor enforced and invalid grips rejected
Motion command rejected by controller	Robot stops; process interruption	6	3	2	36	Invalid or inconsistent motion command format	Command validation and structured message formatting ensure consistent communication with controller
Trajectory exits workspace boundary	Motion infeasible; execution halted	6	4	3	72	Target pose near or beyond reachable workspace limits	Workspace clamping applied to all trajectory points; limits enforced on radial and vertical reach
Inverse kinematics failure due to excessive attachments	Robot cannot compute valid configuration	5	2	3	30	Multiple simultaneous object attachments causing solver overload	Strict attach-detach discipline enforced; each attachment preceded by release of previous objects
Self-collision in simulation	Invalid motion path generated without warning	4	3	5	60	Lack of self-collision detection in simulation environment	Three-phase trajectory structure reduces collision risk; limitation acknowledged for physical deployment
Configuration error (invalid or missing parameters)	System fails at startup; no execution	3	3	1	9	Missing or incorrectly defined configuration parameters	Configuration validation performed at startup; errors detected before motion execution

5.12.2 Mitigation Strategies

Several design measures are incorporated to reduce the likelihood and impact of these failures. Grip stability is ensured through controlled attachment sequences and orientation constraints. Trajectory feasibility is maintained through workspace validation and structured motion planning. Singularity risks are reduced by limiting orientation changes during lateral motion and separating rotational adjustments from the final descent phase.

The flip workflow introduces controlled attachment and detachment stages to ensure correct reorientation of the sheet. Additionally, the use of live object tracking for pose acquisition improves positioning accuracy during both bending and unloading operations. The deterministic execution pipeline further reduces uncertainty by enforcing a fixed sequence of validated operations.

5.12.3 Discussion

The FMEA results indicate that the most critical risks are associated with pose accuracy and trajectory feasibility, as reflected by higher RPN values. These risks are mitigated through a combination of constraint-based planning, controlled motion generation, and staged execution.

Overall, the system demonstrates a balanced design in which high-severity failure modes are either low in occurrence or effectively mitigated through detection and control mechanisms. This supports reliable operation of the robotic bending process while maintaining safety and repeatability.

The presented analysis highlights that the integration of planning, trajectory generation, and execution control not only enables automated bending but also contributes significantly to system robustness. This confirms the effectiveness of the proposed system architecture in handling complex multi-stage manipulation tasks.

Bibliography

[1] *Robots and robotic devices — vocabulary*, International Organization for Standardization, 2012.

[2] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2005, ISBN: 978-0-13-123629-5.

[3] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*, 2nd ed. London: Springer, 2016, ISBN: 978-1-4471-1500-0.

[4] T. Lozano-Pérez, "Robot programming," *Proceedings of the IEEE*, vol. 71, no. 7, pp. 821–841, 1983. DOI: 10.1109/PROC.1983.12678

[5] J. Krüger, T. Krüger, et al., "Skill-based programming of industrial robots," *CIRP Annals*, vol. 68, no. 2, pp. 661–684, 2019. DOI: 10.1016/j.cirp.2019.05.002

[6] G. Pahl, W. Beitz, J. Feldhusen, and K. H. Grote, *Engineering Design: A Systematic Approach*, 3rd ed. London: Springer, 2007. DOI: 10.1007/978-1-84628-319-2

[7] Acieta, *Robotic press brake, welding & palletizing cell for cotterman*, 2018. [Online]. Available: https://www.acieta.com/automation-application/robotically-integrated-bending-solutions-ribs/%20(Accessed:%2027%20March%202026)

[8] S. Aomura and K. Koguchi, "Development of a robotic bending system for sheet metal," *Journal of Materials Processing Technology*, vol. 130–131, pp. 362–367, 2002. DOI: 10.1016/S0924-0136(02)00744-1

[9] S. K. Gupta, D. A. Bourne, K. Kim, and S. Krishnan, "Automated process planning for sheet metal bending operations," *Journal of Manufacturing Systems*, vol. 17, no. 5, pp. 338–360, 1998. DOI: 10.1016/S0278-6125(98)80035-2

[10] L. Biagiotti and C. Melchiorri, *Trajectory Planning for Automatic Machines and Robots*. Springer, 2008.

[11] W. Wan and K. Harada, "Regrasp planning using 10,000s of grasps," *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2017.

[12] J. R. Duflou, D. Van Oudheusden, J.-P. Kruth, and D. Cattrysse, "Modeling of vacuum grippers for the design of energy-efficient vacuum-based handling processes," *CIRP Annals*, vol. 55, no. 1, pp. 17–20, 2006. DOI: 10.1016/S0007-8506(07)60375-0

[13] R. Bogue, "Growth in industrial robot applications," *Industrial Robot: An International Journal*, vol. 45, no. 1, pp. 1–6, 2018. DOI: 10.1108/IR-10-2017-0199

[14] RoboDK, *Robodk: Simulation and offline programming software for industrial robots*, https://robodk.com, Accessed: 27 March 2026, 2024.

[15] G. Chryssolouris, *Manufacturing Systems: Theory and Practice*. Springer, 2013.

[16] F. Tao, Q. Qi, et al., "Digital twin driven smart manufacturing," *Journal of Manufacturing Systems*, vol. 48, pp. 157–169, 2019. DOI: 10.1016/j.jmsy.2018.05.003

[17] V. Villani, F. Pini, F. Leali, and C. Secchi, "Survey on human–robot collaboration in industrial settings," *Mechatronics*, vol. 55, pp. 248–266, 2018. DOI: 10.1016/j.mechatronics.2018.02.009

[18] J. Bohg, A. Morales, T. Asfour, and D. Kragic, "Data-driven grasp synthesis—a survey," *IEEE Transactions on Robotics*, 2014.

[19] P. Neto, "Off-line programming and simulation from cad drawings: Robot-assisted sheet metal bending," in *Proceedings (conference not explicitly specified)*, Department of Mechanical Engineering, CEMUC, Coimbra, Portugal, 2013.

[20] drag&bot GmbH. "Anyrobot interface documentation." Ulm, Germany, Accessed: Mar. 2, 2026. [Online]. Available: https://docs.dragandbot.com/anyrobot-interface

[21] Z. Pan, J. Polden, N. Larkin, S. van Duin, and J. Norrish, "Recent progress on programming methods for industrial robots," *Robotics and Computer-Integrated Manufacturing*, vol. 20, no. 2, pp. 137–150, 2012. DOI: 10.1016/j.rcim.2003.07.004

[22] K. Thramboulidis, "Model-driven engineering for industrial automation systems: A survey," *IEEE Transactions on Industrial Informatics*, vol. 9, no. 4, pp. 1893–1903, 2013. DOI: 10.1109/TII.2013.2251233

[23] M. Bär, G. Reinhart, and J. Schilp, "Virtual commissioning of robotic systems: A review and outlook," *Procedia CIRP*, vol. 72, pp. 129–134, 2018.

[24] M. Quigley et al., "ROS: an Open-Source Robot Operating System," in *Proceedings of the ICRA Workshop on Open Source Software*, Kobe, Japan, 2009.

[25] Python Software Foundation, *Python 3 language reference*, Accessed: 2026-03-02, 2024. [Online]. Available: https://docs.python.org/3/

[26] F. Tao, M. Zhang, Y. Liu, and A. Nee, "Digital twin in industry: State-of-the-art," *IEEE Transactions on Industrial Informatics*, vol. 17, no. 4, pp. 2400–2415, 2021. DOI: 10.1109/TII.2020.3009610

[27] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022. DOI: 10.1126/scirobotics.abm6074

[28] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017, ISBN: 9781107156302.

[29] A. Bicchi and V. Kumar, "Robotic grasping and contact: A review," *ICRA*, 2000.

[30] Schmalz GmbH, *Vacuum handling systems: Design principles and force calculation*, Technical reference for vacuum gripping force models, 2020.

[31] K. Shoemake, "Animating rotation with quaternion curves," in *Proceedings of the 12th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '85)*, ACM, 1985, pp. 245–254. DOI: 10.1145/325334.325242

[32] T. Flash and N. Hogan, "The coordination of arm movements: An experimentally confirmed mathematical model," *The Journal of Neuroscience*, vol. 5, no. 7, pp. 1688–1703, 1985. DOI: 10.1523/JNEUROSCI.05-07-01688.1985

[33] A. e. a. Gasparetto, "Path planning and trajectory planning algorithms: A general overview," *Mechanism and Machine Theory*, 2012.